

Practical 1 — Airline Passengers

Practical 1A

Introduction to Time Series Analysis with R

1. Importing data files with RStudio & preliminary manipulations.

As first thing in this practical, we will import a time series data set from RStudio itself, and we will see how to create the same time series manually step-by-step, by using the “ts” function from the “tseries” package.

```
#1. Importing data files with RStudio & preliminary manipulations.  
# Import AirPassengers file from RStudio data  
  
library(readxl)  
library(tseries)  
  
data("AirPassengers")  
AP<- AirPassengers  
print("Time series from the AirPassenger dataset (TIME SERIES AP):")
```

```
## [1] "Time series from the AirPassenger dataset (TIME SERIES AP):"
```

```
AP
```

```
##      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec  
## 1949 112 118 132 129 121 135 148 148 136 119 104 118  
## 1950 115 126 141 135 125 149 170 170 158 133 114 140  
## 1951 145 150 178 163 172 178 199 199 184 162 146 166  
## 1952 171 180 193 181 183 218 230 242 209 191 172 194  
## 1953 196 196 236 235 229 243 264 272 237 211 180 201  
## 1954 204 188 235 227 234 264 302 293 259 229 203 229  
## 1955 242 233 267 269 270 315 364 347 312 274 237 278  
## 1956 284 277 317 313 318 374 413 405 355 306 271 306  
## 1957 315 301 356 348 355 422 465 467 404 347 305 336  
## 1958 340 318 362 348 363 435 491 505 404 359 310 337  
## 1959 360 342 406 396 420 472 548 559 463 407 362 405  
## 1960 417 391 419 461 472 535 622 606 508 461 390 432
```

```
cat("\n")
```

```
cat("Class type of the latter time series:", class(AP), "\n")
```

```
## Class type of the latter time series: ts
```

```
# Save number of passengers in vector x and date (time) in vector t
x <- as.numeric(AP)
t <- time(AP)

cat("\n")
```

```
# Create a time series using the "tseries" package, with periodicity 12 months (frequency=12) and start
# We will use the same numbers of passengers as given in the AirPassenger data (i.e. "x")
xt<-ts(x,start=1949,frequency=12)
print("Time series created manually, using passengers numbers from AirPassenger dataset (TIME SERIES xt.
```

```
## [1] "Time series created manually, using passengers numbers from AirPassenger dataset (TIME SERIES x
```

```
xt
```

```
##      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
## 1949 112 118 132 129 121 135 148 148 136 119 104 118
## 1950 115 126 141 135 125 149 170 170 158 133 114 140
## 1951 145 150 178 163 172 178 199 199 184 162 146 166
## 1952 171 180 193 181 183 218 230 242 209 191 172 194
## 1953 196 196 236 235 229 243 264 272 237 211 180 201
## 1954 204 188 235 227 234 264 302 293 259 229 203 229
## 1955 242 233 267 269 270 315 364 347 312 274 237 278
## 1956 284 277 317 313 318 374 413 405 355 306 271 306
## 1957 315 301 356 348 355 422 465 467 404 347 305 336
## 1958 340 318 362 348 363 435 491 505 404 359 310 337
## 1959 360 342 406 396 420 472 548 559 463 407 362 405
## 1960 417 391 419 461 472 535 622 606 508 461 390 432
```

```
cat("\n")
```

```
cat("Class type of the latter time series:", class(xt), "\n")
```

```
## Class type of the latter time series: ts
```

```
# xt and AP are two identical time series, just created in two different ways.
```

As expected, both time series “AP” and “xt” are the same, both in terms of time, frequency, passengers and class types (they both are class type ts). Let’s now explore the summary statistics of the time series “AP”:

```
summary(AP)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   104.0   180.0   265.5   280.3   360.5   622.0
```

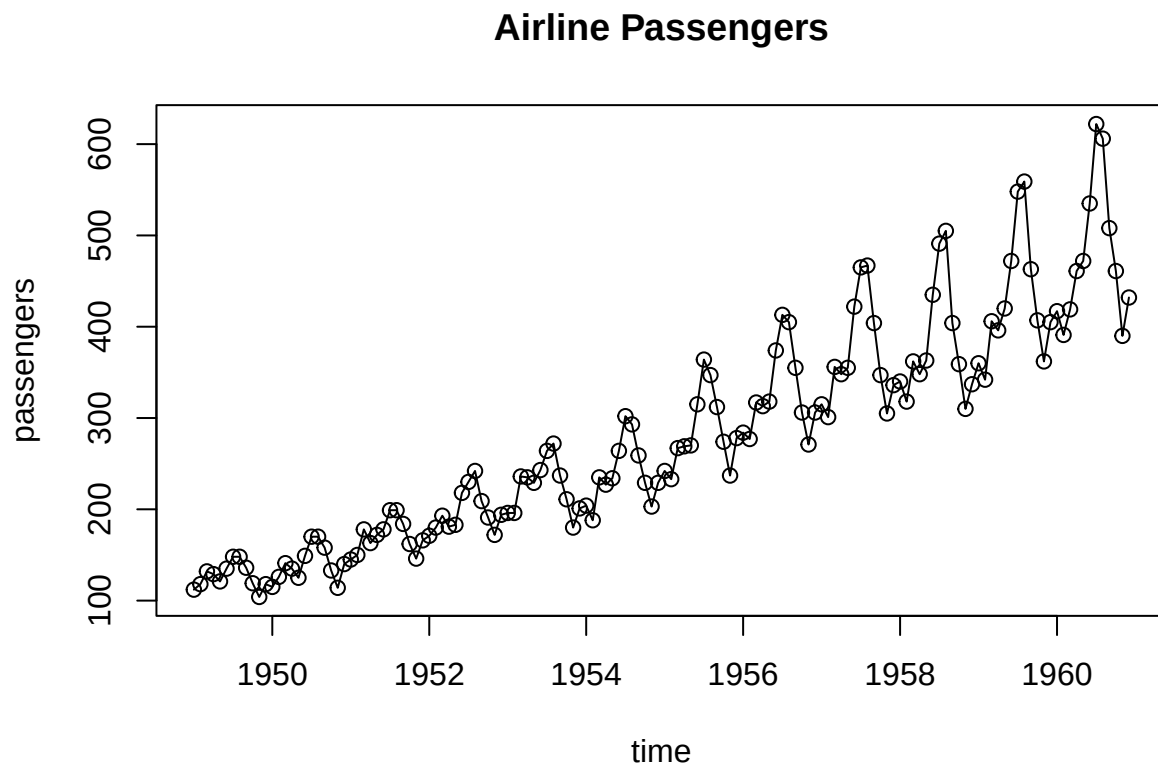
These numbers are coherent with the plot below, titled “Airline Passengers”, where indeed minimum values are around 100 passengers (104) and maximum are around 600 passengers (622).

2. Graphical representation

We will see different options to plot the time series:

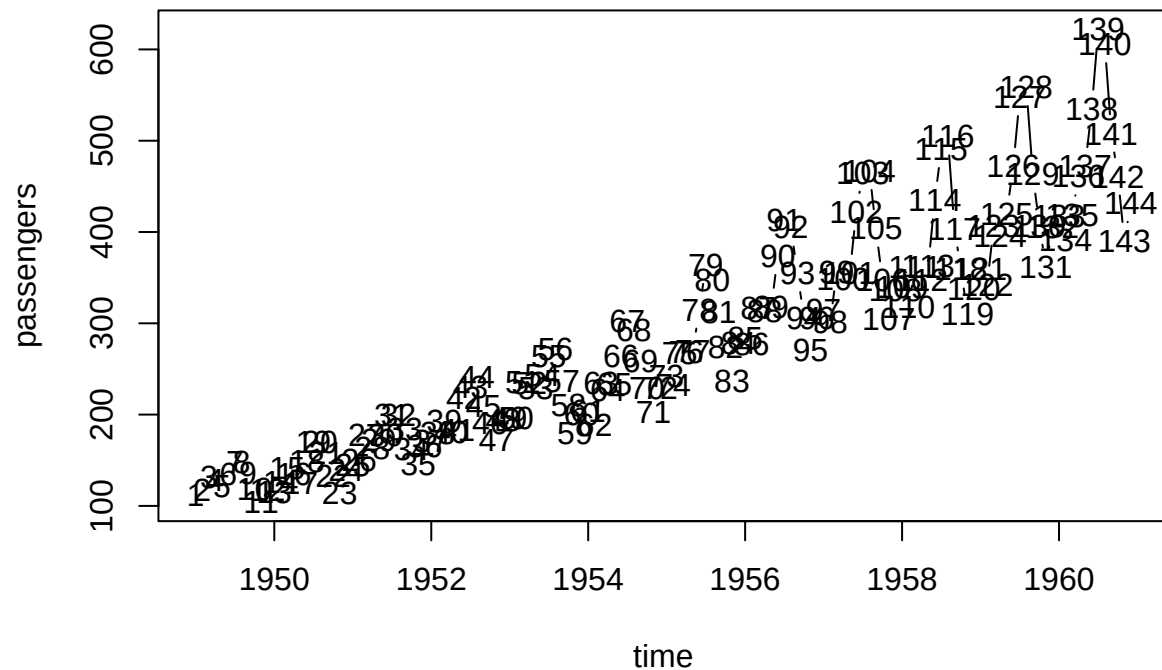
- we will try the two cases suggested by the practical (represent datapoints with circles and connect them with lines, or represent datapoints with a number representing the n. of passengers)
- and we will experiment few more options discovered through the “help” function of “plot”, such as: histogram, only points, stair steps (discretized view)

```
# Option 1 for plotting: lines connected through circles  
# In this case first argument is time series as one unique element (xt)  
plot(xt, main="Airline Passengers", xlab="time", ylab="passengers", type="o")
```



```
# Option 2 for plotting: lines connected through numbers  
# In this case the time series is composed by the vectors x and t, so both have to be given as first arguments  
# time on x-axis (so t is the 1° argument) and number of passengers on y-axis (so x is the 2° argument)  
plot(t, x, main="Airline Passengers", xlab="time", ylab="passengers", type="l")
```

Airline Passengers

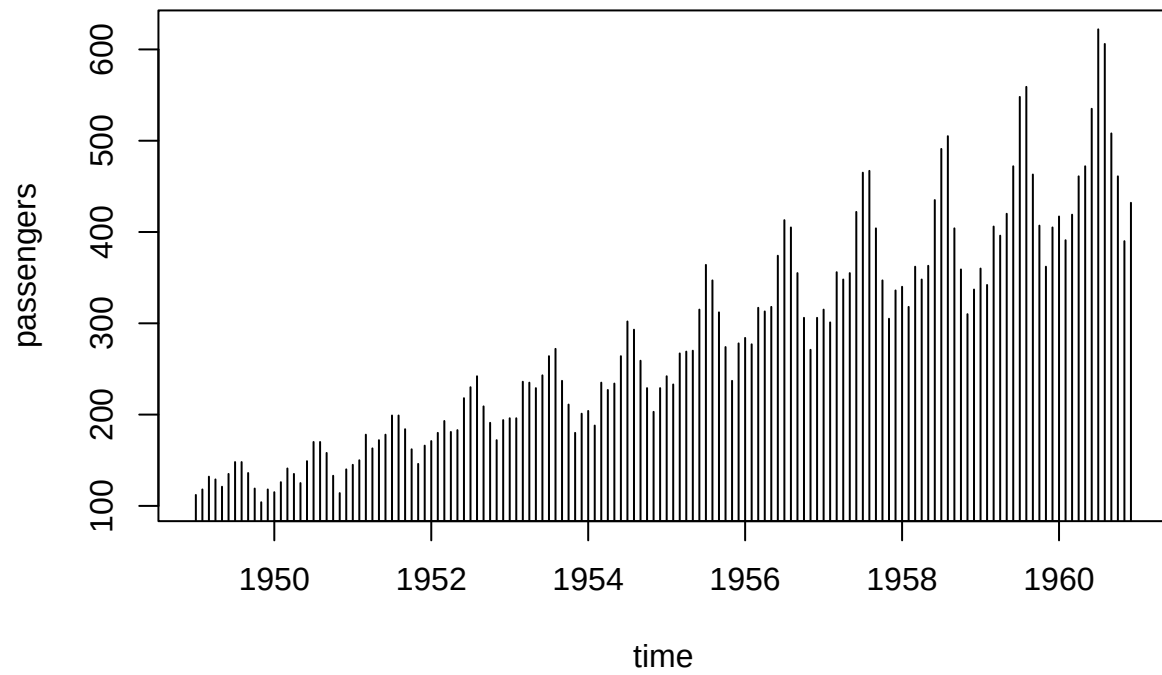


```
# Check the help for the "plot" function, in particular the different options for "type".
help("plot")
```

```
## Help on topic 'plot' was found in the following packages:
##
##   Package      Library
##   graphics    /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library
##   base         /Library/Frameworks/R.framework/Resources/library
##
##
## Using the first match ...
```

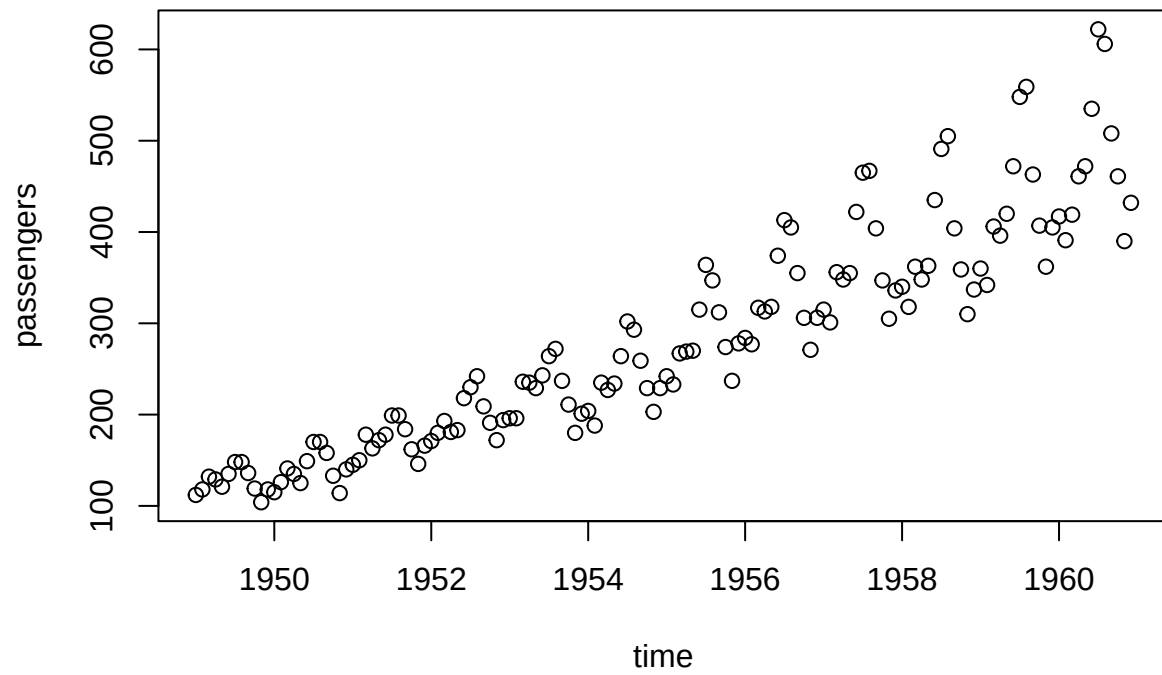
```
# I will try to plot the histogram
plot(xt, main="Airline Passengers", xlab="time", ylab="passengers", type="h")
```

Airline Passengers



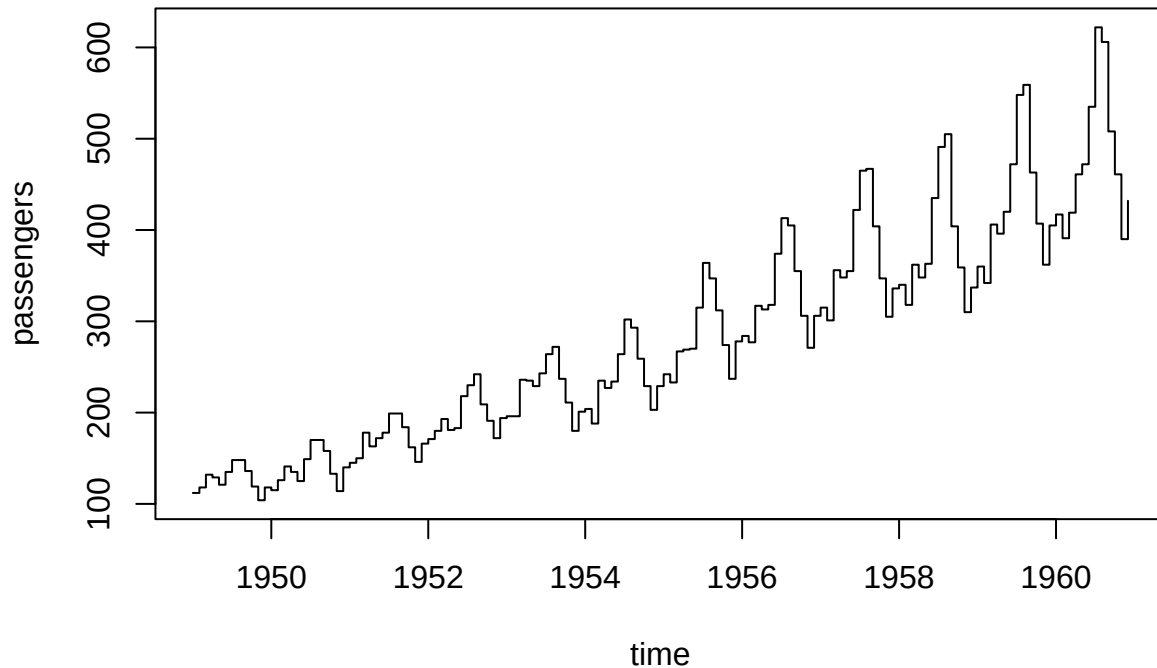
```
# Plot only the points  
plot(xt, main="Airline Passengers", xlab="time", ylab="passengers", type="p")
```

Airline Passengers



```
# Plot stair steps  
plot(xt, main="Airline Passengers", xlab="time", ylab="passengers", type="s")
```

Airline Passengers



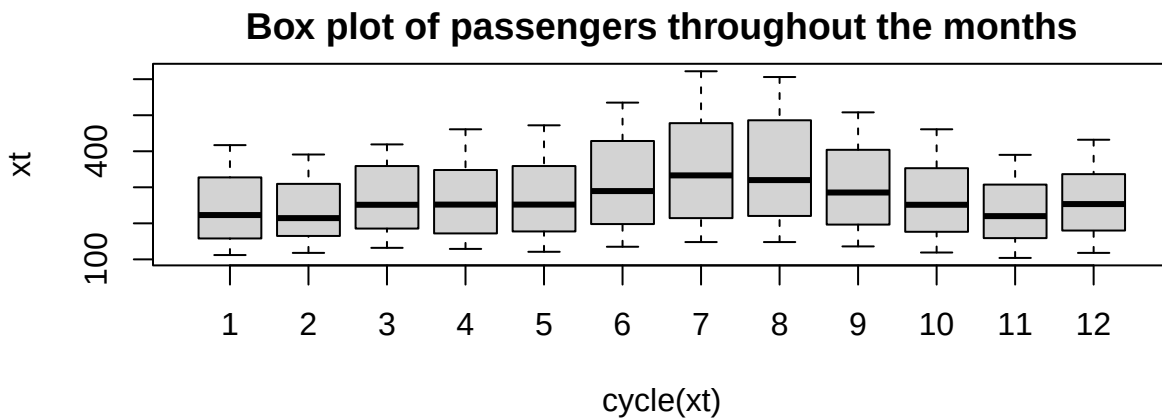
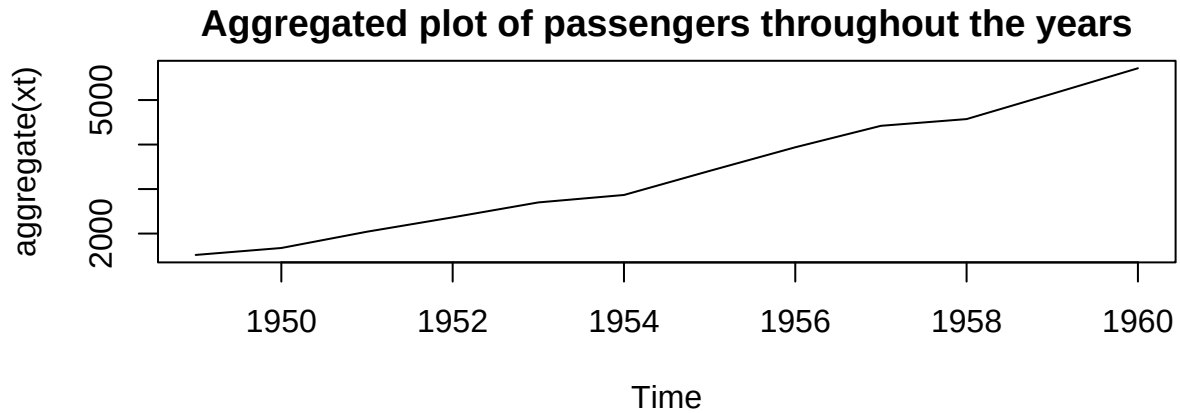
In this case, we can observe that the series shows an overall increasing trend over the years. However, it may not always be trivial to see the global trend - as time series data are often characterized by fluctuations, short-term increases and decreases, and irregular spikes.

For this purpose, we use the aggregate function: it summarizes (with a sum function) the amount of passengers per year. This results in a much smoother plot, allowing us to visualize the underlying trend more clearly.

Furthermore, it is useful to understand how values typically vary across months within a year. For this, we use the cycle function: it groups the number of passengers by month, so that we can easily visualize the average number of passengers per month and their range of observed values. This will be shown in the boxplot below.

```
# Another interesting option is to use "aggregate" and "cycle" to make a yearly summary:
layout(1:2, heights = c(4, 4)) # we add "heights" parameter so that the scale of y-axis is bigger and
par(mar = c(4, 5, 2, 1)) # more room for y-axis labels
plot(aggregate(xt), main="Aggregated plot of passengers throughout the years") #aggregate collapses the
#for example with monthly & yearly data like in AirPassengers dataset, it groups al

boxplot(xt-cycle(xt), main="Box plot of passengers throughout the months") #Create box plots of the tim
```



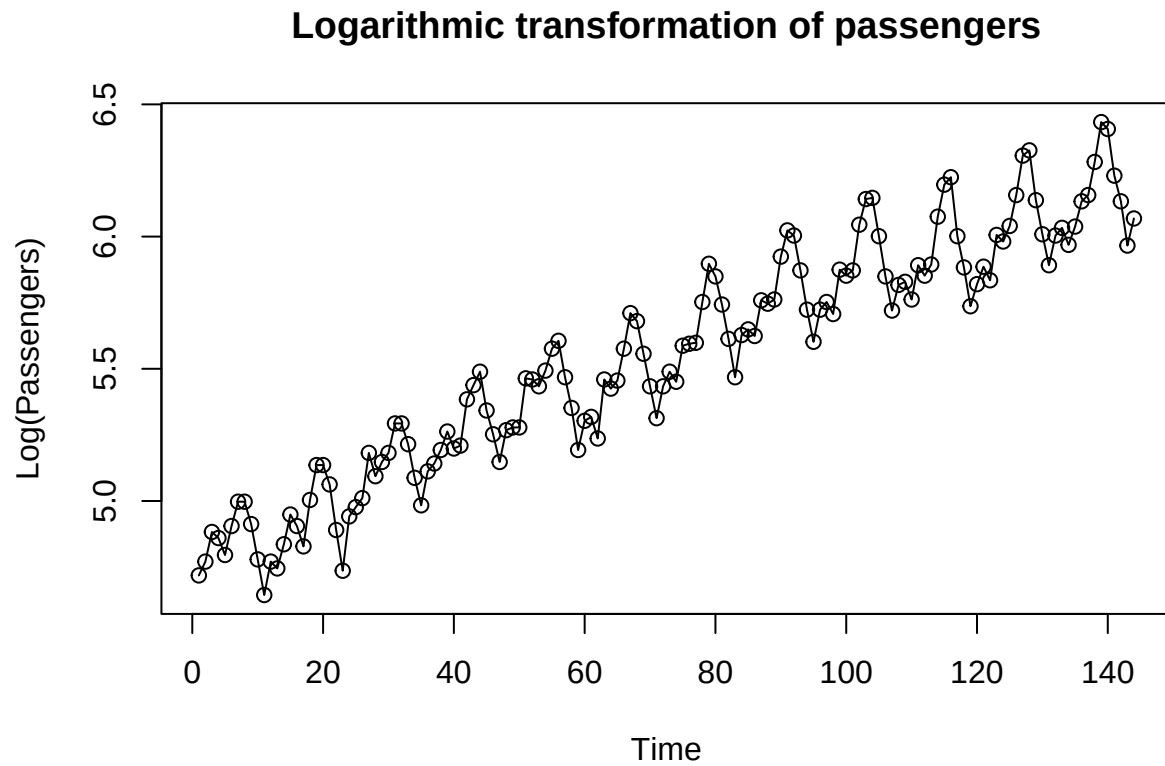
3. Transforming a time series

We are now going to apply a logarithmic transformation to the series, to stabilize the variance.

```
y<-log(x)
y
```

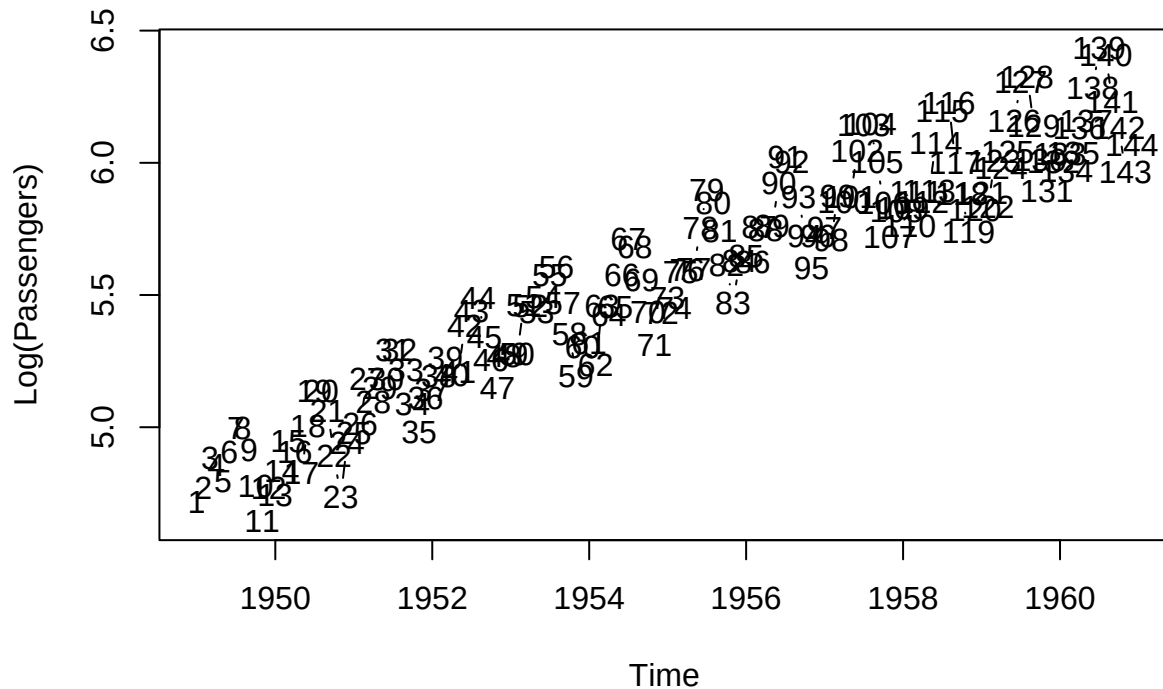
```
## [1] 4.718499 4.770685 4.882802 4.859812 4.795791 4.905275 4.997212 4.997212
## [9] 4.912655 4.779123 4.644391 4.770685 4.744932 4.836282 4.948760 4.905275
## [17] 4.828314 5.003946 5.135798 5.135798 5.062595 4.890349 4.736198 4.941642
## [25] 4.976734 5.010635 5.181784 5.093750 5.147494 5.181784 5.293305 5.293305
## [33] 5.214936 5.087596 4.983607 5.111988 5.141664 5.192957 5.262690 5.198497
## [41] 5.209486 5.384495 5.438079 5.488938 5.342334 5.252273 5.147494 5.267858
## [49] 5.278115 5.278115 5.463832 5.459586 5.433722 5.493061 5.575949 5.605802
## [57] 5.468060 5.351858 5.192957 5.303305 5.318120 5.236442 5.459586 5.424950
## [65] 5.455321 5.575949 5.710427 5.680173 5.556828 5.433722 5.313206 5.433722
## [73] 5.488938 5.451038 5.587249 5.594711 5.598422 5.752573 5.897154 5.849325
## [81] 5.743003 5.613128 5.468060 5.627621 5.648974 5.624018 5.758902 5.746203
## [89] 5.762051 5.924256 6.023448 6.003887 5.872118 5.723585 5.602119 5.723585
## [97] 5.752573 5.707110 5.874931 5.852202 5.872118 6.045005 6.142037 6.146329
## [105] 6.001415 5.849325 5.720312 5.817111 5.828946 5.762051 5.891644 5.852202
## [113] 5.894403 6.075346 6.196444 6.224558 6.001415 5.883322 5.736572 5.820083
## [121] 5.886104 5.834811 6.006353 5.981414 6.040255 6.156979 6.306275 6.326149
## [129] 6.137727 6.008813 5.891644 6.003887 6.033086 5.968708 6.037871 6.133398
## [137] 6.156979 6.282267 6.432940 6.406880 6.230481 6.133398 5.966147 6.068426
```

```
plot (y, main = "Logarithmic transformation of passengers", xlab="Time", ylab="Log(Passengers)",type="o")
```



```
# I will plot also omitting the "t" so that all the numbers disappear, to have a clearer view of the tr
plot (t, y, main = "Logarithmic transformation of passengers", xlab="Time", ylab="Log(Passengers)",type="o")
```

Logarithmic transformation of passengers

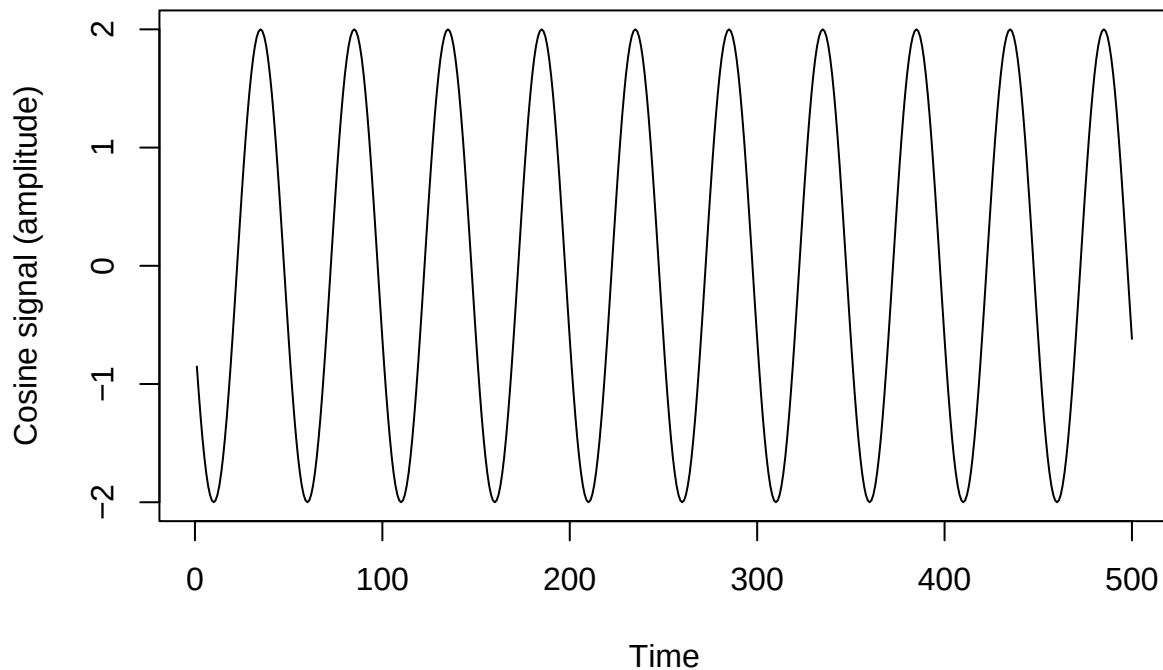


Observations:

1. In the *Airline Passenger plot* above, at the beginning of this practical, the variability was large and increasing over time: the peaks get bigger as the time increases. Which means that the **variance is not constant**, it's a non-stationary variance. In other words, the early years fluctuate a bit, while the later years fluctuate much more.
2. In the *Logarithmic transformation of passengers plot*, we can observe that the variability is smaller: while the upward trend is still present, the fluctuations look more uniform across time. Which means that the **variance is more stable**.

As the practical suggest, we now explore how powerful R is in representing functions. For example:

```
t2<-1:500
c<-2*cos(2*pi*t2/50+0.6*pi)
plot(t2, c, type="l", xlab="Time", ylab="Cosine signal (amplitude)")
```



We defined the expression as a cosine signal with amplitude 2, period 50 time units, and a phase shift of 0.6π . With R it's extremely easy to plot this, and observe that it's a periodic time series with constant variance.

4. Estimating a linear trend.

In the code above we have defined two objects, `t` and `x` (time and airline passengers respectively). We will try, in the next block, to obtain the best linear approximation of our data:

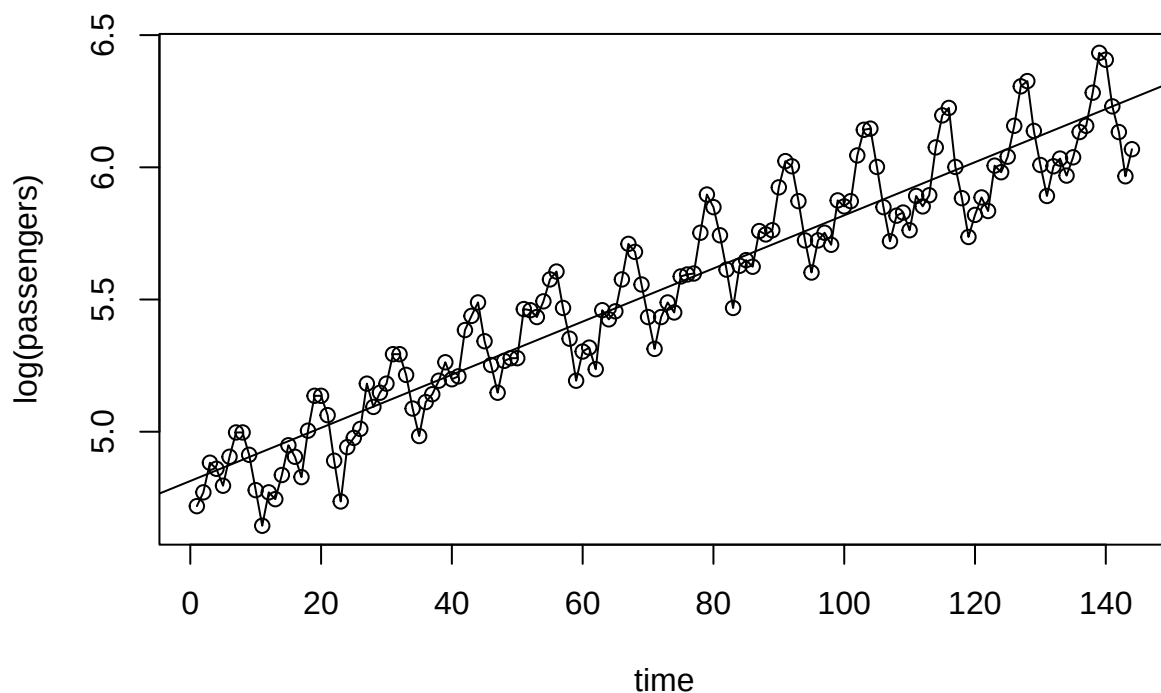
```
t3<-1:144 # create a simple time index:
line=lm(y~t3) # fits a linear regression model (fit the best straight line through the data)
summary(line)
```

```
##
## Call:
## lm(formula = y ~ t3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.30858 -0.10388 -0.01796  0.09738  0.29538
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.8136683   0.0232940   206.65  <2e-16 ***
## t3           0.0100484   0.0002787    36.05  <2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.139 on 142 degrees of freedom
## Multiple R-squared:  0.9015, Adjusted R-squared:  0.9008
## F-statistic: 1300 on 1 and 142 DF,  p-value: < 2.2e-16
```

```
plot(t3, y, main="Airline Passengers (logarithmic transformation) and fit linear approximation", type="o",
abline(line) #Adds the fitted regression line on top of the plot
```

Airline Passengers (logarithmic transformation) and fit linear approximation



```
#To extract the trend to the logarithmic data we can do:
```

```
y<-log(x)
lm(y~t3)
```

```
##
## Call:
## lm(formula = y ~ t3)
##
## Coefficients:
## (Intercept)          t3
##    4.81367      0.01005
```

```
trend<-0.01005*t3+4.81367
trend
```

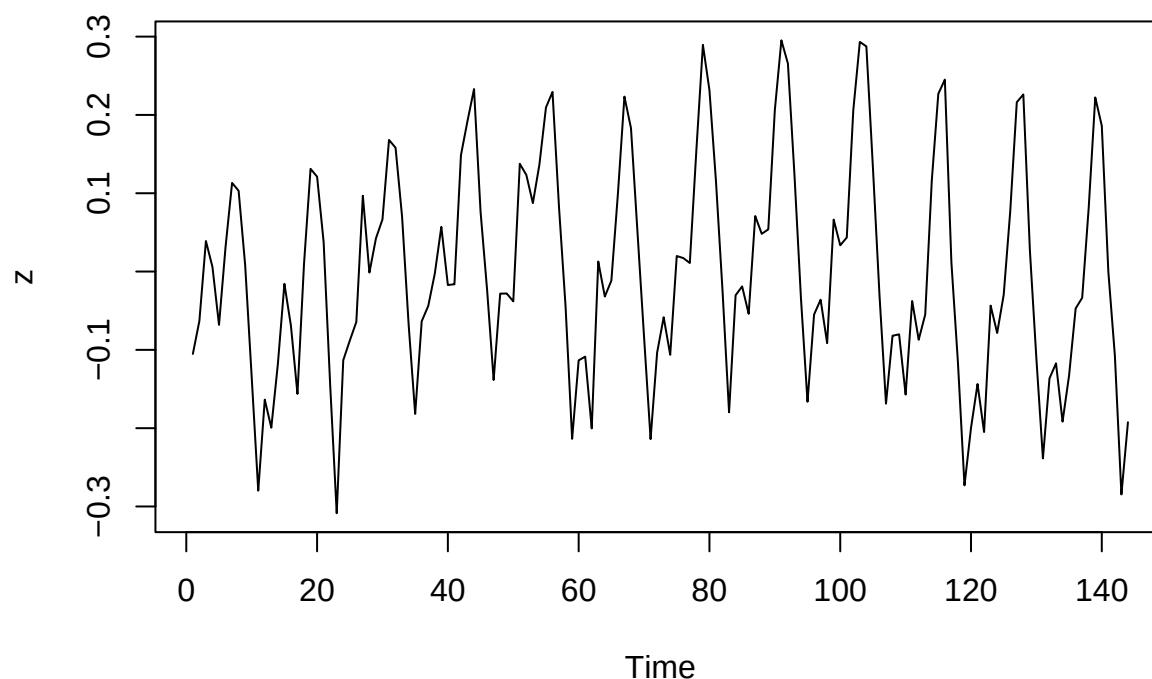
```
## [1] 4.82372 4.83377 4.84382 4.85387 4.86392 4.87397 4.88402 4.89407 4.90412
## [10] 4.91417 4.92422 4.93427 4.94432 4.95437 4.96442 4.97447 4.98452 4.99457
## [19] 5.00462 5.01467 5.02472 5.03477 5.04482 5.05487 5.06492 5.07497 5.08502
## [28] 5.09507 5.10512 5.11517 5.12522 5.13527 5.14532 5.15537 5.16542 5.17547
## [37] 5.18552 5.19557 5.20562 5.21567 5.22572 5.23577 5.24582 5.25587 5.26592
## [46] 5.27597 5.28602 5.29607 5.30612 5.31617 5.32622 5.33627 5.34632 5.35637
## [55] 5.36642 5.37647 5.38652 5.39657 5.40662 5.41667 5.42672 5.43677 5.44682
## [64] 5.45687 5.46692 5.47697 5.48702 5.49707 5.50712 5.51717 5.52722 5.53727
## [73] 5.54732 5.55737 5.56742 5.57747 5.58752 5.59757 5.60762 5.61767 5.62772
## [82] 5.63777 5.64782 5.65787 5.66792 5.67797 5.68802 5.69807 5.70812 5.71817
## [91] 5.72822 5.73827 5.74832 5.75837 5.76842 5.77847 5.78852 5.79857 5.80862
## [100] 5.81867 5.82872 5.83877 5.84882 5.85887 5.86892 5.87897 5.88902 5.89907
## [109] 5.90912 5.91917 5.92922 5.93927 5.94932 5.95937 5.96942 5.97947 5.98952
## [118] 5.99957 6.00962 6.01967 6.02972 6.03977 6.04982 6.05987 6.06992 6.07997
## [127] 6.09002 6.10007 6.11012 6.12017 6.13022 6.14027 6.15032 6.16037 6.17042
## [136] 6.18047 6.19052 6.20057 6.21062 6.22067 6.23072 6.24077 6.25082 6.26087
```

```
z<-y-trend #z=residuals of the time series after log transforming and extracting the linear trend.
z
```

```
## [1] -0.1052211287 -0.0630853755 0.0389819226 0.0059424044 -0.0681294544
## [6] 0.0313047784 0.1131922738 0.1031422738 0.0085348857 -0.1350465069
## [11] -0.2798291009 -0.1635853755 -0.1993878716 -0.1180880930 -0.0156601096
## [16] -0.0691952216 -0.1562062627 0.0093763059 0.1311784371 0.1211284371
## [21] 0.0378750330 -0.1444208718 -0.3086215516 -0.1132275774 -0.0881862576
## [26] -0.0643347059 0.0967635503 -0.0013197992 0.0423744768 0.0666135503
## [31] 0.1680848247 0.1580348247 0.0696157576 -0.0677736648 -0.1818133783
## [36] -0.0634822116 -0.0438564435 -0.0026131491 0.0570701889 -0.0171729687
## [41] -0.0162338472 0.1487250628 0.1922593089 0.2330677262 0.0764142520
## [46] -0.0236965720 -0.1385255232 -0.0282118409 -0.0280053408 -0.0380553408
## [51] 0.1376118050 0.1233155141 0.0874020036 0.1366914433 0.2095291031
## [56] 0.2293320663 0.0815401411 -0.0447118665 -0.2136631491 -0.1133650919
## [61] -0.1086000062 -0.2003280372 0.0127655141 -0.0319199825 -0.0115988846
## [66] 0.0989791031 0.2234070174 0.1831026090 0.0497080617 -0.0834479964
## [71] -0.2140140210 -0.1035479964 -0.0583822738 -0.1063315464 0.0198286584
## [76] 0.0172413796 0.0109019590 0.1550026388 0.2895338676 0.2316547799
## [81] 0.1152831878 -0.0246418936 -0.1797598589 -0.0302488863 -0.0189457618
## [86] -0.0539524938 0.0708817739 0.0481331905 0.0539313828 0.2060857974
## [91] 0.2952275930 0.2656170671 0.1237977895 -0.0347848980 -0.1663011791
## [96] -0.0548848980 -0.0359473612 -0.0914597353 0.0663107309 0.0335324798
## [101] 0.0433977895 0.2062353140 0.2932174056 0.2874592577 0.1324948780
## [106] -0.0296452201 -0.1687082234 -0.0819588400 -0.0801743824 -0.1571186172
## [111] -0.0375757882 -0.0870675202 -0.0549171657 0.1159760311 0.2270241278
## [116] 0.2450884293 0.0118948780 -0.1162476115 -0.2730477025 -0.1995870696
## [121] -0.1436159685 -0.2049592629 -0.0434668404 -0.0784557887 -0.0296652887
## [126] 0.0770089856 0.2162552869 0.2260794732 0.0276070541 -0.1113568146
## [131] -0.2385757882 -0.1363829329 -0.1172337782 -0.1916624400 -0.1325490801
## [136] -0.0470719570 -0.0335410144 0.0816967469 0.2223200927 0.1862099861
## [141] -0.0002385524 -0.1073719570 -0.2846732609 -0.1924444118
```

```
plot.ts(z, main="Residuals from fitting linearly the Logarithmic Airline Passengers")
```

Residuals from fitting linearly the Logarithmic Airline Passengers



In plot “**Airline Passengers (logarithmic transformation) and fit linear approximation**” we represented the logarithmic time series with the line that best fits it.

Then, we represented the line as: $trend = 0.01005 * t3 + 4.81367$ and computed the residuals:

- **At each time point:** $residual = actual\ value - predicted\ trend\ value$

Eventually we plot the residuals: this allows us to observe the following aspects, which are otherwise not explained by the trend: seasonality, short-term fluctuations, noise.

5. Estimating the seasonal component

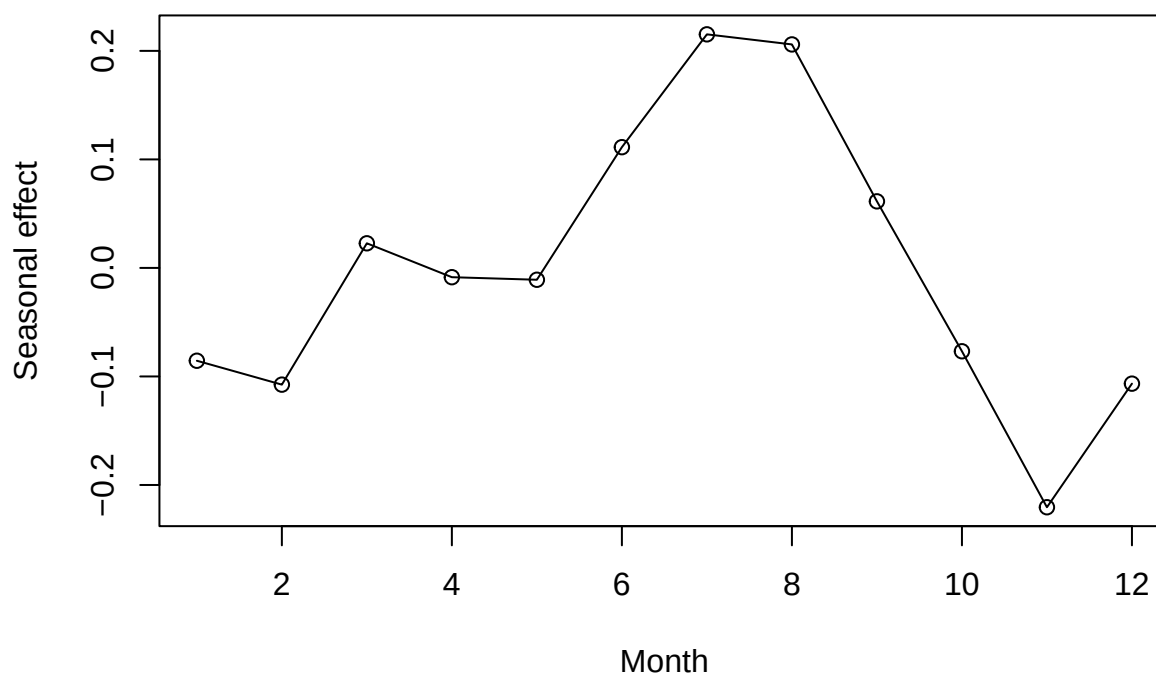
```
# To eliminate seasonality of z we can do:
dim(z)<-c(12,12)
z
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] -0.105221129 -0.199387872 -0.088186258 -0.043856443 -0.02800534
## [2,] -0.063085376 -0.118088093 -0.064334706 -0.002613149 -0.03805534
## [3,]  0.038981923 -0.015660110  0.096763550  0.057070189  0.13761181
## [4,]  0.005942404 -0.069195222 -0.001319799 -0.017172969  0.12331551
## [5,] -0.068129454 -0.156206263  0.042374477 -0.016233847  0.08740200
## [6,]  0.031304778  0.009376306  0.066613550  0.148725063  0.13669144
## [7,]  0.113192274  0.131178437  0.168084825  0.192259309  0.20952910
```

```
## [8,] 0.103142274 0.121128437 0.158034825 0.233067726 0.22933207
## [9,] 0.008534886 0.037875033 0.069615758 0.076414252 0.08154014
## [10,] -0.135046507 -0.144420872 -0.067773665 -0.023696572 -0.04471187
## [11,] -0.279829101 -0.308621552 -0.181813378 -0.138525523 -0.21366315
## [12,] -0.163585376 -0.113227577 -0.063482212 -0.028211841 -0.11336509
##      [,6]      [,7]      [,8]      [,9]     [,10]     [,11]
## [1,] -0.10860001 -0.05838227 -0.01894576 -0.03594736 -0.08017438 -0.14361597
## [2,] -0.20032804 -0.10633155 -0.05395249 -0.09145974 -0.15711862 -0.20495926
## [3,] 0.01276551 0.01982866 0.07088177 0.06631073 -0.03757579 -0.04346684
## [4,] -0.03191998 0.01724138 0.04813319 0.03353248 -0.08706752 -0.07845579
## [5,] -0.01159888 0.01090196 0.05393138 0.04339779 -0.05491717 -0.02966529
## [6,] 0.09897910 0.15500264 0.20608580 0.20623531 0.11597603 0.07700899
## [7,] 0.22340702 0.28953387 0.29522759 0.29321741 0.22702413 0.21625529
## [8,] 0.18310261 0.23165478 0.26561707 0.28745926 0.24508843 0.22607947
## [9,] 0.04970806 0.11528319 0.12379779 0.13249488 0.01189488 0.02760705
## [10,] -0.08344800 -0.02464189 -0.03478490 -0.02964522 -0.11624761 -0.11135681
## [11,] -0.21401402 -0.17975986 -0.16630118 -0.16870822 -0.27304770 -0.23857579
## [12,] -0.10354800 -0.03024889 -0.05488490 -0.08195884 -0.19958707 -0.13638293
##      [,12]
## [1,] -0.1172337782
## [2,] -0.1916624400
## [3,] -0.1325490801
## [4,] -0.0470719570
## [5,] -0.0335410144
## [6,] 0.0816967469
## [7,] 0.2223200927
## [8,] 0.1862099861
## [9,] -0.0002385524
## [10,] -0.1073719570
## [11,] -0.2846732609
## [12,] -0.1924444118

#We have converted the series in a matrix where rows are data of the same month.
#Now we compute monthly row means, the global mean and the seasonal component.
e<-apply(z,1,mean)
ee<-mean(e)
s<-e-ee
plot(s,main = "Seasonal components per month", ylab="Seasonal effect", xlab="Month", type="o")
```

Seasonal components per month



#We have now captured the seasonal component. Now we do:

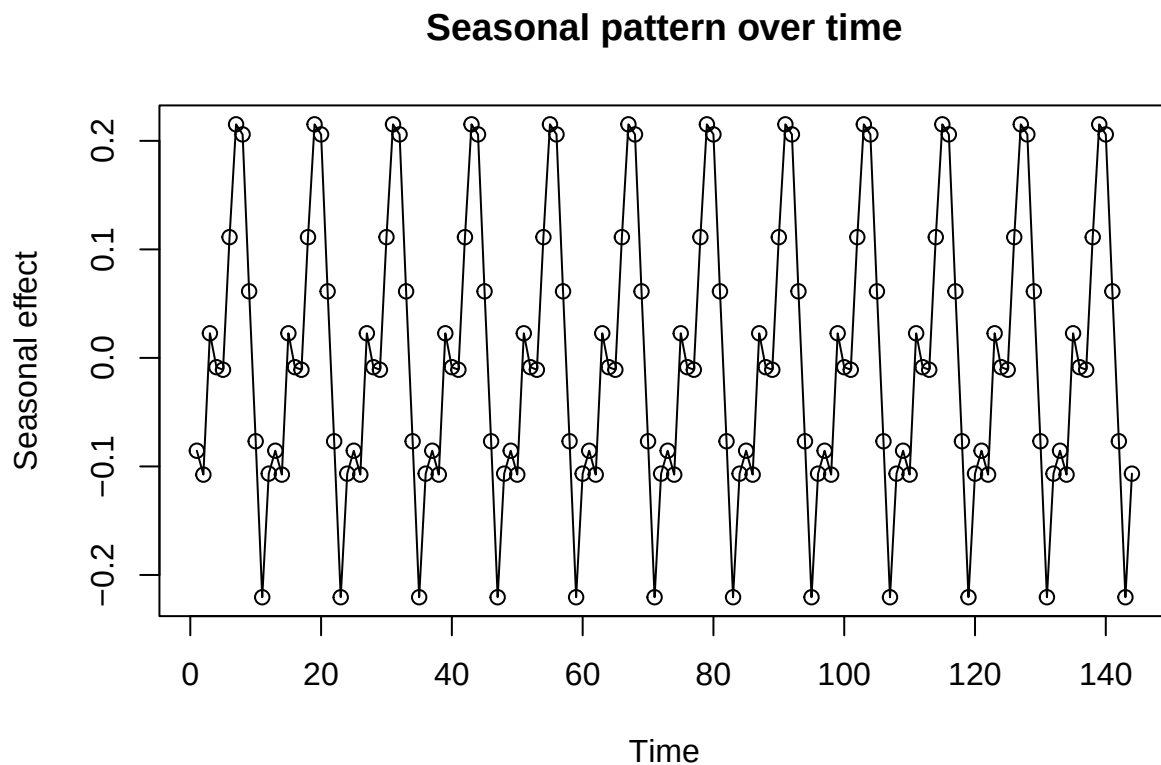
```
est<-array(s,144)
```

```
est
```

```
## [1] -0.085510673 -0.107546692 0.022699235 -0.008550814 -0.010904651
## [6] 0.111260355 0.215221486 0.205945452 0.061329655 -0.220508687
## [11] -0.106624886 -0.085510673 -0.107546692 0.022699235
## [16] -0.008550814 -0.010904651 0.111260355 0.215221486 0.205945452
## [21] 0.061329655 -0.220508687 -0.106624886 -0.085510673
## [26] -0.107546692 0.022699235 -0.008550814 -0.010904651 0.111260355
## [31] 0.215221486 0.205945452 0.061329655 -0.220508687
## [36] -0.106624886 -0.085510673 -0.107546692 0.022699235 -0.008550814
## [41] -0.010904651 0.111260355 0.215221486 0.205945452 0.061329655
## [46] -0.076809781 -0.220508687 -0.106624886 -0.085510673 -0.107546692
## [51] 0.022699235 -0.008550814 -0.010904651 0.111260355 0.215221486
## [56] 0.205945452 0.061329655 -0.220508687 -0.106624886
## [61] -0.085510673 -0.107546692 0.022699235 -0.008550814 -0.010904651
## [66] 0.111260355 0.215221486 0.205945452 0.061329655 -0.220508687
## [71] -0.106624886 -0.085510673 -0.107546692 0.022699235
## [76] -0.008550814 -0.010904651 0.111260355 0.215221486 0.205945452
## [81] 0.061329655 -0.220508687 -0.106624886 -0.085510673
## [86] -0.107546692 0.022699235 -0.008550814 -0.010904651 0.111260355
## [91] 0.215221486 0.205945452 0.061329655 -0.220508687
## [96] -0.106624886 -0.085510673 -0.107546692 0.022699235 -0.008550814
## [101] -0.010904651 0.111260355 0.215221486 0.205945452 0.061329655
## [106] -0.076809781 -0.220508687 -0.106624886 -0.085510673 -0.107546692
```

```
## [111]  0.022699235 -0.008550814 -0.010904651  0.111260355  0.215221486
## [116]  0.205945452  0.061329655 -0.076809781 -0.220508687 -0.106624886
## [121] -0.085510673 -0.107546692  0.022699235 -0.008550814 -0.010904651
## [126]  0.111260355  0.215221486  0.205945452  0.061329655 -0.076809781
## [131] -0.220508687 -0.106624886 -0.085510673 -0.107546692  0.022699235
## [136] -0.008550814 -0.010904651  0.111260355  0.215221486  0.205945452
## [141]  0.061329655 -0.076809781 -0.220508687 -0.106624886
```

```
plot(est,main = "Seasonal pattern over time", ylab = "Seasonal effect", xlab = "Time",type="o")
```

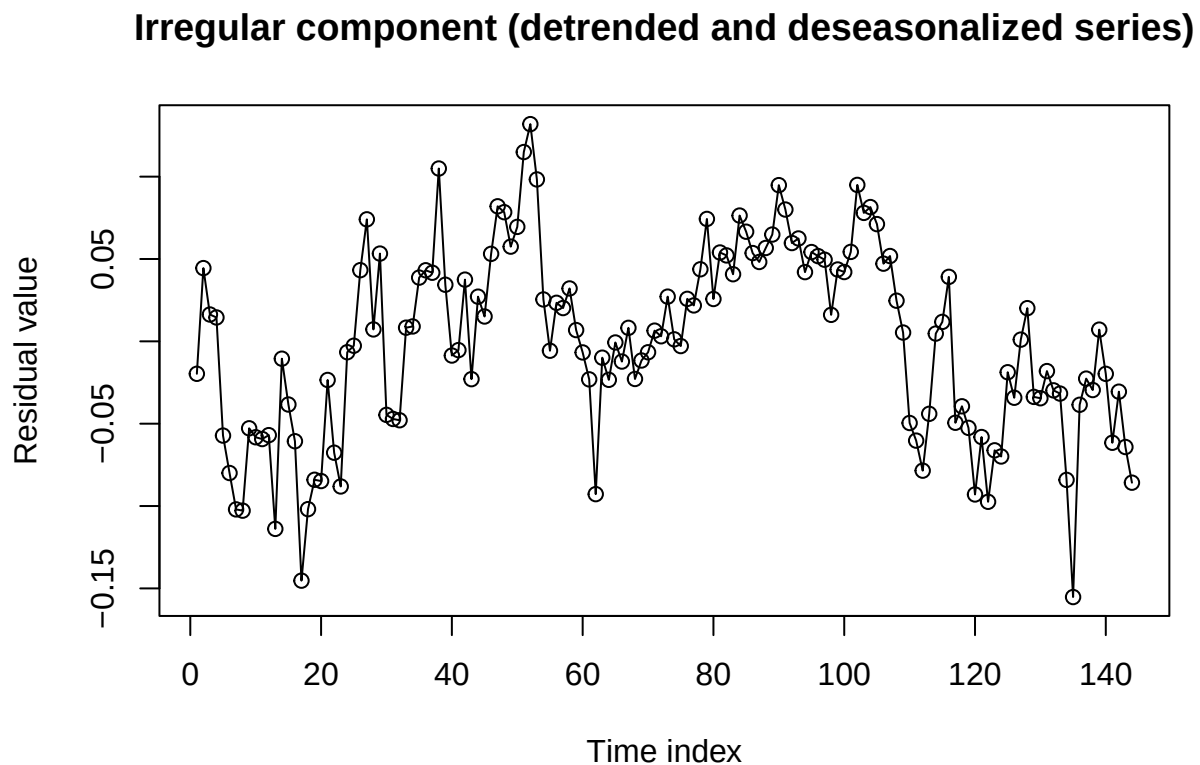


```
dim(z)<-144
u<-z-est
u
```

```
## [1] -0.0197104556  0.0444613161  0.0162826873  0.0144932187 -0.0572248037
## [6] -0.0799555762 -0.1020292126 -0.1028031786 -0.0527947696 -0.0582367256
## [11] -0.0593204142 -0.0569604893 -0.1138771986 -0.0105414014 -0.0383593449
## [16] -0.0606444072 -0.1453016120 -0.1018840487 -0.0840430493 -0.0848170153
## [21] -0.0234546223 -0.0676110905 -0.0881128650 -0.0066026911 -0.0026755845
## [26]  0.0432119857  0.0740643150  0.0072310151  0.0532791275 -0.0446468043
## [31] -0.0471366616 -0.0479106276  0.0082861023  0.0090361165  0.0386953083
## [36]  0.0431426746  0.0416542296  0.1049335425  0.0343709536 -0.0086221544
## [41] -0.0053291964  0.0374647082 -0.0229621774  0.0271222738  0.0150845967
## [46]  0.0531132093  0.0819831634  0.0784130453  0.0575053323  0.0694913509
## [51]  0.1149125697  0.1318663285  0.0983066543  0.0254310887 -0.0056923832
```

```
## [56] 0.0233866140 0.0202104858 0.0320979148 0.0068455375 -0.0067402057
## [61] -0.0230893331 -0.0927813455 -0.0099337211 -0.0233691682 -0.0006942339
## [66] -0.0122812515 0.0081855310 -0.0228428433 -0.0116215936 -0.0066382152
## [71] 0.0064946657 0.0030768898 0.0271283992 0.0012151452 -0.0028705769
## [76] 0.0257921939 0.0218066097 0.0437422842 0.0743123813 0.0257093276
## [81] 0.0539535325 0.0521678877 0.0407488277 0.0763759999 0.0665649112
## [86] 0.0535941978 0.0481825386 0.0566840049 0.0648360335 0.0948254428
## [91] 0.0800061066 0.0596716148 0.0624681342 0.0420248832 0.0542075075
## [96] 0.0517399882 0.0495633119 0.0160869564 0.0436114956 0.0420832941
## [101] 0.0543024402 0.0949749594 0.0779959192 0.0815138053 0.0711652227
## [106] 0.0471645612 0.0518004632 0.0246660462 0.0053362907 -0.0495719256
## [111] -0.0602750235 -0.0785167059 -0.0440125150 0.0047156765 0.0118026414
## [116] 0.0391429770 -0.0494347773 -0.0394378302 -0.0525390159 -0.0929621834
## [121] -0.0581052955 -0.0974125713 -0.0661660757 -0.0699049744 -0.0187606380
## [126] -0.0342513690 0.0010338006 0.0201340208 -0.0337226012 -0.0345470333
## [131] -0.0180671016 -0.0297580466 -0.0317231051 -0.0841157484 -0.1552483154
## [136] -0.0385211427 -0.0226363637 -0.0295636077 0.0070986064 -0.0197354663
## [141] -0.0615682077 -0.0305621757 -0.0641645743 -0.0858195255
```

```
plot(u, main="Irregular component (detrended and deseasonalized series)", xlab="Time index", ylab="Residual value")
```



The plots above show: 1. Seasonal component over 12 months 2. Seasonal component over time (144 time units) 3. Irregular component (de-trended and de-seasonalized series)

The plots 1 and 2 show values around 0, specifically between approximately -0.2 and 0.2. This is due to the fact that the **seasonal component** “s” has been centered. In fact, it is obtained by subtracting the global mean from each monthly average: $s = e - \bar{e}$.

Then, the **vector est** is obtained by repeating the *seasonal component* “*s*” over the entire time period (144 time units) so that it matches the length of the series. As a consequence, also the plot of *est* (2nd plot) is centered around 0, with values between approximately -0.2 and 0.2.

The third plot shows the **irregular component “u”**, which also has negative and positive values around 0 but, contrarily to the other plots, it doesn’t show any trend or seasonal pattern. In fact, *u* represents the residuals after removing trend and seasonality, and it’s therefore the noise component of the time series ($u = z - \text{est}$).

6. Filtering

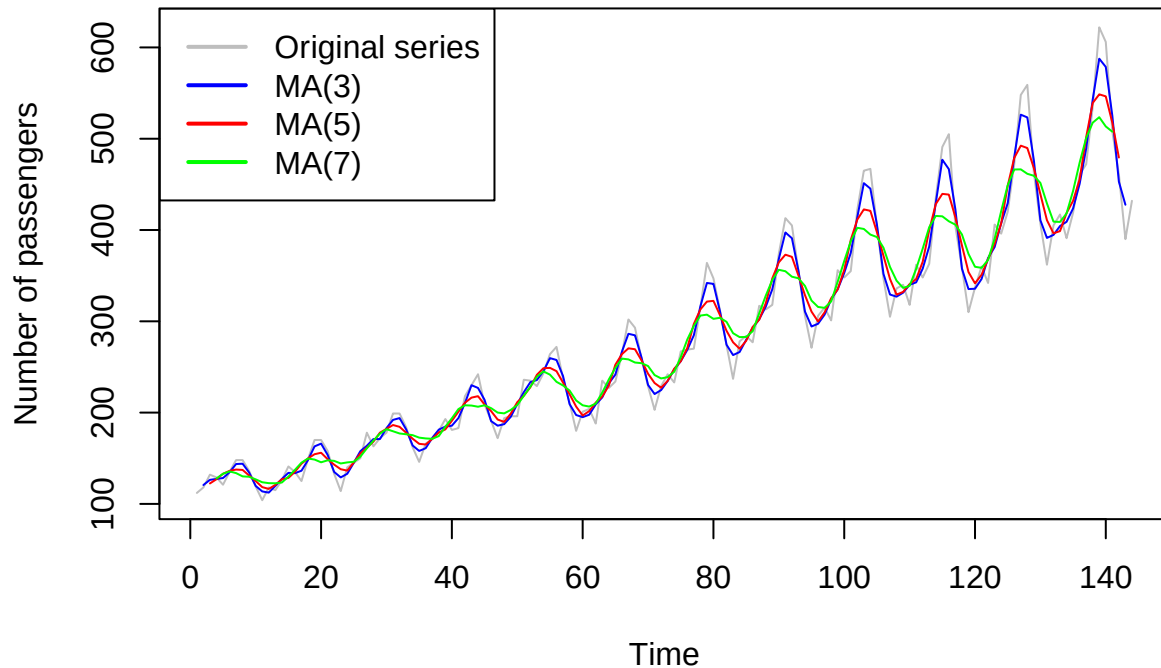
```
#Given x, the following instruction applies a moving average of order q=1 to the series:
v3<-filter(x, sides=2,rep(1,3)/3) #It replaces each observation with the average of its neighbors. So n

v5<-filter(x, sides=2, rep(1,5)/5)
v7<-filter(x, sides=2, rep(1,7)/7)

plot(x, type="l", main = "Moving average smoothing", xlab="Time", ylab="Number of passengers", col="grey",
lines(v3, col="blue")
lines(v5, col="red")
lines(v7, col="green")

legend("topleft",
      legend = c("Original series",
                  "MA(3)",
                  "MA(5)",
                  "MA(7)"),
      col = c("grey","blue","red","green"),
      lty = 1,
      lwd = 2)
```

Moving average smoothing



What we represent in the plot:

- MA(3) → moving average with window size 3
- MA(5) → window size 5
- MA(7) → window size 7

Observations:

The moving averages make the time series smoother, by reducing the fluctuations. In fact, the moving average replaces each observation with the average of its neighbors (3, 5 or 7 neighbors depending on the size of the window).

In this way the noise is “averaged out” for adjacent time points, and hence the filtered time series is smoother, with less noise and it highlights the underlying trend of the series more clearly.

On the other side, larger windows also mean compromising with less visibility into short-term changes.

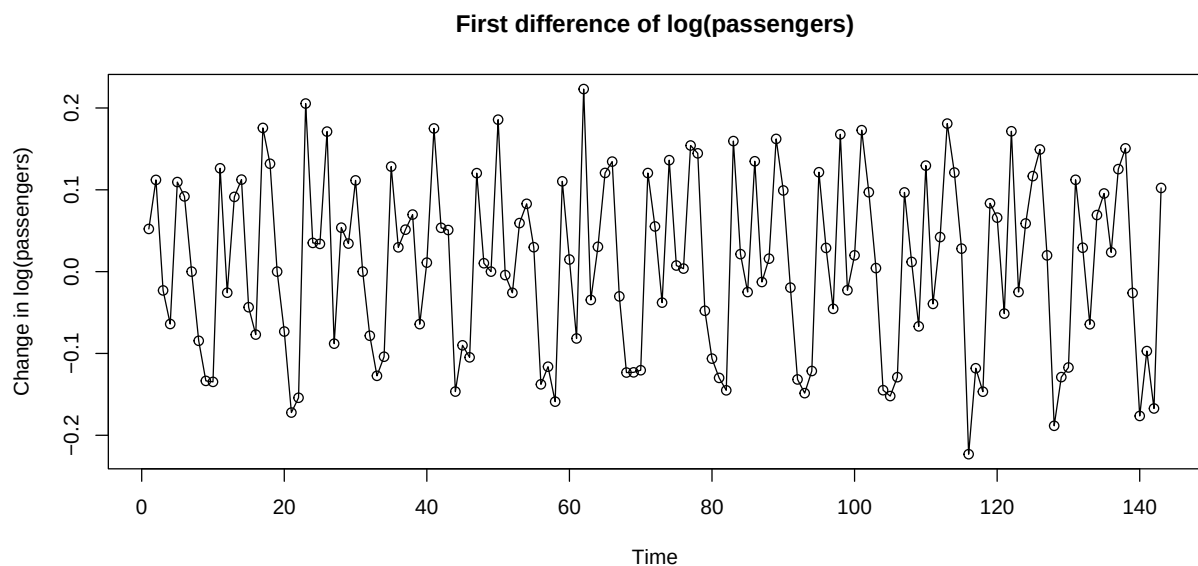
7. Differentiation

```
y<-log(x)
dy<-diff(y) #first order difference
dy
```

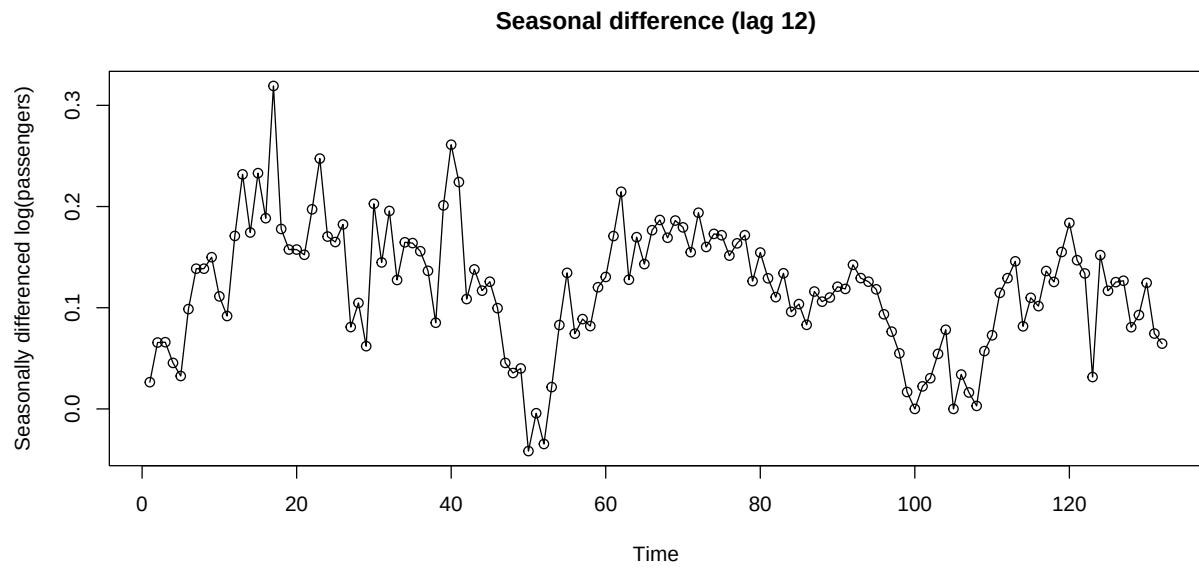
```
##      [1]  0.052185753  0.112117298 -0.022989518 -0.064021859  0.109484233
##      [6]  0.091937495  0.000000000 -0.084557388 -0.133531393 -0.134732594
```

```
## [11] 0.126293725 -0.025752496 0.091349779 0.112477983 -0.043485112
## [16] -0.076961041 0.175632569 0.131852131 0.000000000 -0.073203404
## [21] -0.172245905 -0.154150680 0.205443974 0.035091320 0.033901552
## [26] 0.171148256 -0.088033349 0.053744276 0.034289073 0.111521274
## [31] 0.000000000 -0.078369067 -0.127339422 -0.103989714 0.128381167
## [36] 0.029675768 0.051293294 0.069733338 -0.064193158 0.010989122
## [41] 0.175008910 0.053584246 0.050858417 -0.146603474 -0.090060824
## [46] -0.104778951 0.120363682 0.010256500 0.000000000 0.185717146
## [51] -0.004246291 -0.025863511 0.059339440 0.082887660 0.029852963
## [56] -0.137741925 -0.116202008 -0.158901283 0.110348057 0.014815086
## [61] -0.081678031 0.223143551 -0.034635497 0.030371098 0.120627988
## [66] 0.134477914 -0.030254408 -0.123344547 -0.123106058 -0.120516025
## [71] 0.120516025 0.055215723 -0.037899273 0.136210205 0.007462721
## [76] 0.003710579 0.154150680 0.144581229 -0.047829088 -0.106321592
## [81] -0.129875081 -0.145067965 0.159560973 0.021353124 -0.024956732
## [86] 0.134884268 -0.012698583 0.015848192 0.162204415 0.099191796
## [91] -0.019560526 -0.131769278 -0.148532688 -0.121466281 0.121466281
## [96] 0.028987537 -0.045462374 0.167820466 -0.022728251 0.019915310
## [101] 0.172887525 0.097032092 0.004291852 -0.144914380 -0.152090098
## [106] -0.129013003 0.096799383 0.011834458 -0.066894235 0.129592829
## [111] -0.039441732 0.042200354 0.180943197 0.121098097 0.028114301
## [116] -0.223143551 -0.118092489 -0.146750091 0.083510633 0.066021101
## [121] -0.051293294 0.171542423 -0.024938948 0.058840500 0.116724274
## [126] 0.149296301 0.019874186 -0.188422419 -0.128913869 -0.117168974
## [131] 0.112242855 0.029199155 -0.064378662 0.069163360 0.095527123
## [136] 0.023580943 0.125287761 0.150673346 -0.026060107 -0.176398538
## [141] -0.097083405 -0.167251304 0.102278849
```

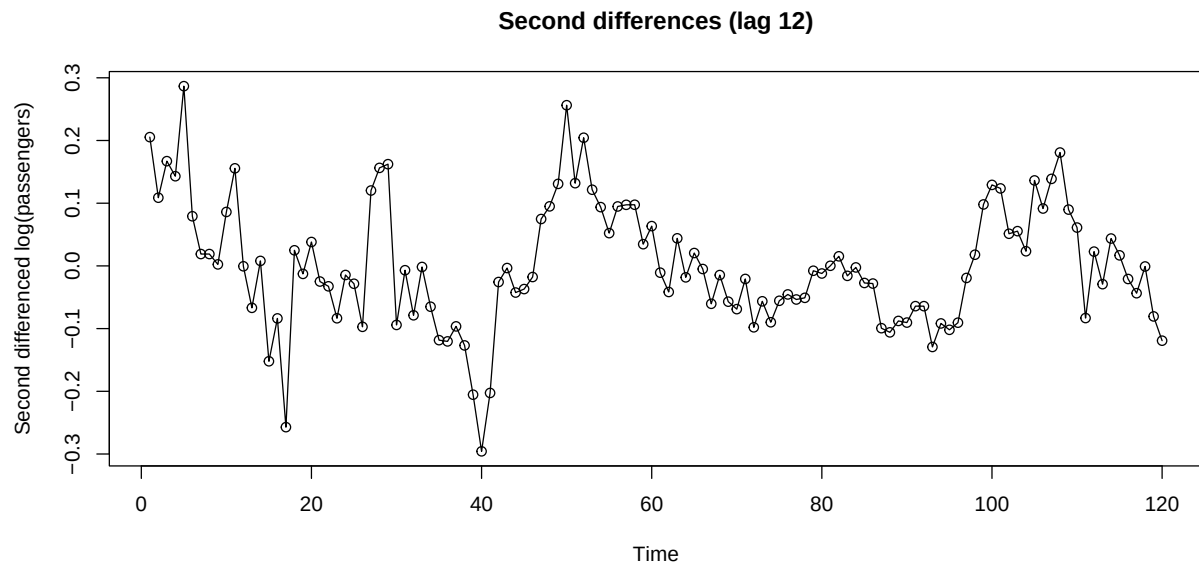
```
plot(dy, type="o", main="First difference of log(passengers)", xlab="Time", ylab="Change in log(passengers)")
```



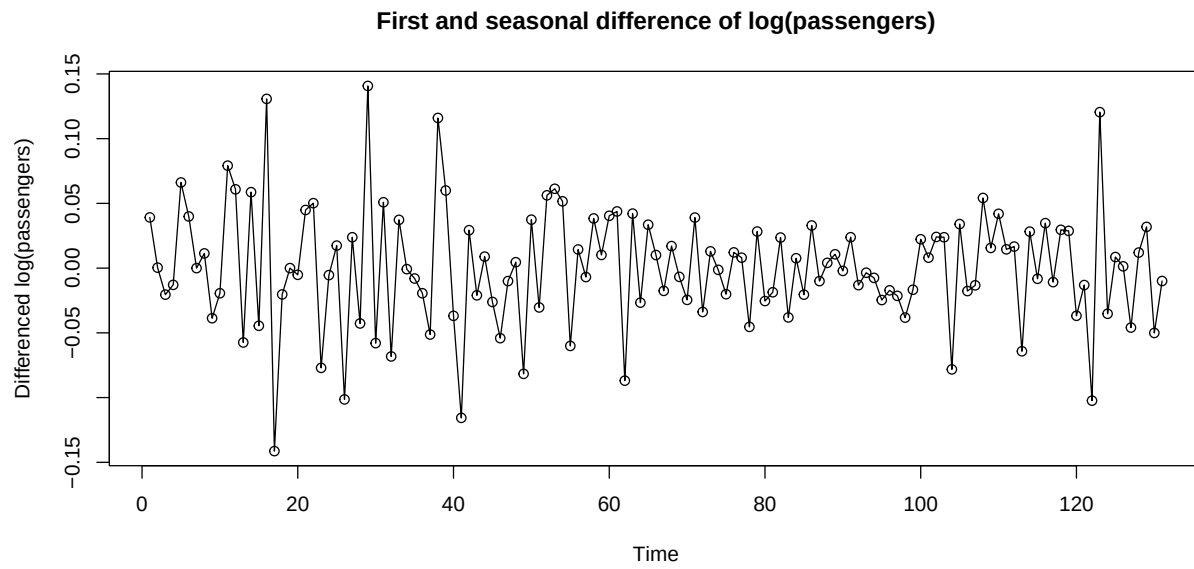
```
de<-diff(y, lag=12) #differences of order 12
plot(de, type="o", main="Seasonal difference (lag 12)", xlab="Time", ylab="Seasonally differenced log(p
```



```
de<-diff(y,12,2) #second differences of order 12
plot(de, type="o", main="Second differences (lag 12)", xlab="Time", ylab="Second differenced log(passengers)")
```



```
y<-log(x)
dy12<-diff(y, lag=12)
#plot (dy12, xlab="time", ylab="passengers", type="o")
ddy12<-diff(dy12)
plot(ddy12, type="o", main="First and seasonal difference of log(passengers)", xlab="Time", ylab="Differenced log(passengers)")
```



In the section above we have divided the differentiation in 3 steps:

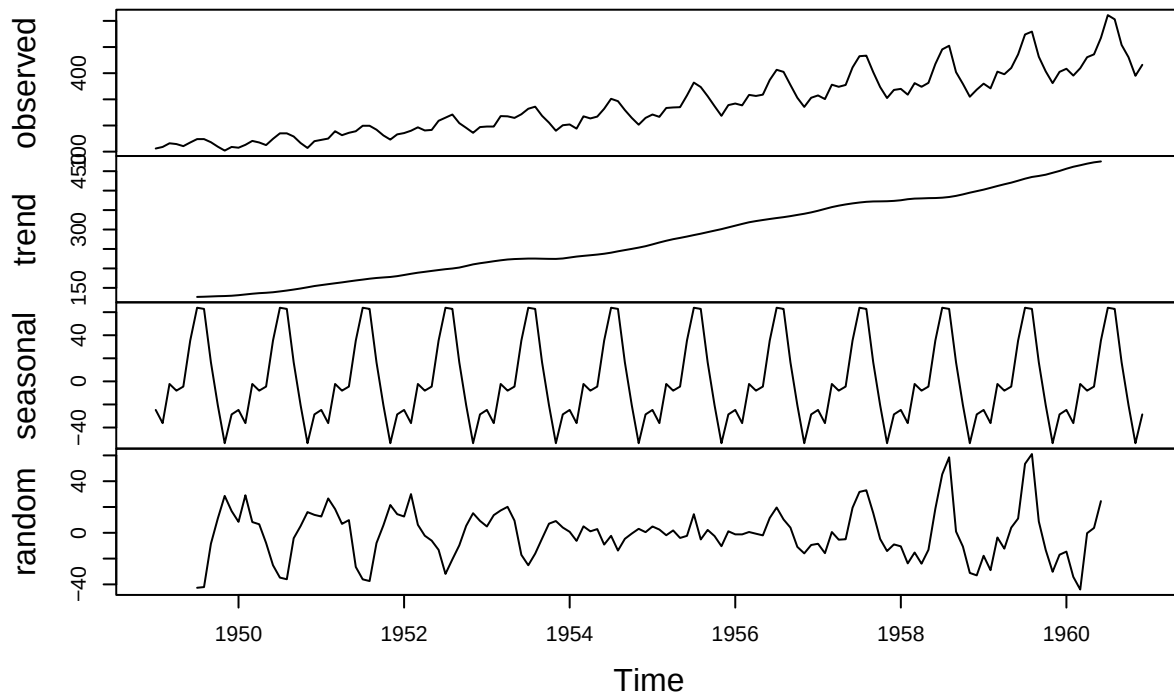
1. **First order differentiation:** for each time point, $\text{diff}(y)$ computes the difference between the value of the time point y_t and the previous y_{t-1} . We see that the linear trend is removed, there is no more an upward trend but rather the series fluctuates around zero. While the seasonality is still visible.
2. **Difference of order 12:** for each time point, $\text{diff}(y, \text{lag}=12)$ computes the difference between the value of the time point y_t and the time point at 12 steps ago y_{t-12} . Since frequency = 12, it compares each month with the same month last year. This removes the yearly seasonal pattern.
3. **Second differences of order 12:** it applies the difference of order 12 twice
4. **Seasonal + first difference:** here we first removed seasonality with a **Difference of order 12**, and then we removed the trend with a **First order differentiation**. In this case we observe a time series that does not have a trend nor seasonality and it fluctuates around zero.

8. Decompositions

The practical says to take the time series xt and decompose it directly using a moving average only, as in the following example:

```
plot (decompose(xt))
```

Decomposition of additive time series

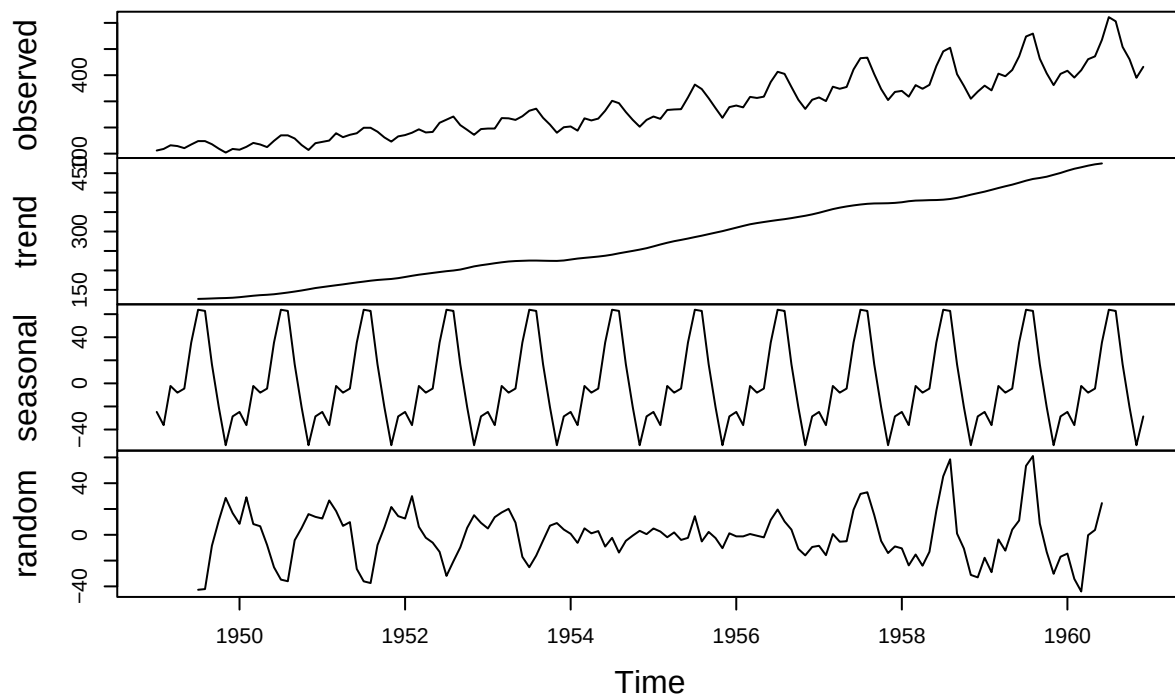


This is a very useful view that already summarizes all the components that earlier we obtained by manually subtracting points of the time series. The “observed” time series is the one that we originally plotted, i.e. the original time series. And it is composed of the other elements plotted below it: **Observed = Trend + Seasonal + Random**. In fact, if we add the parameter `type="additive"` as in paragraph below, it will generate the same plot.

Now we can extract the components:

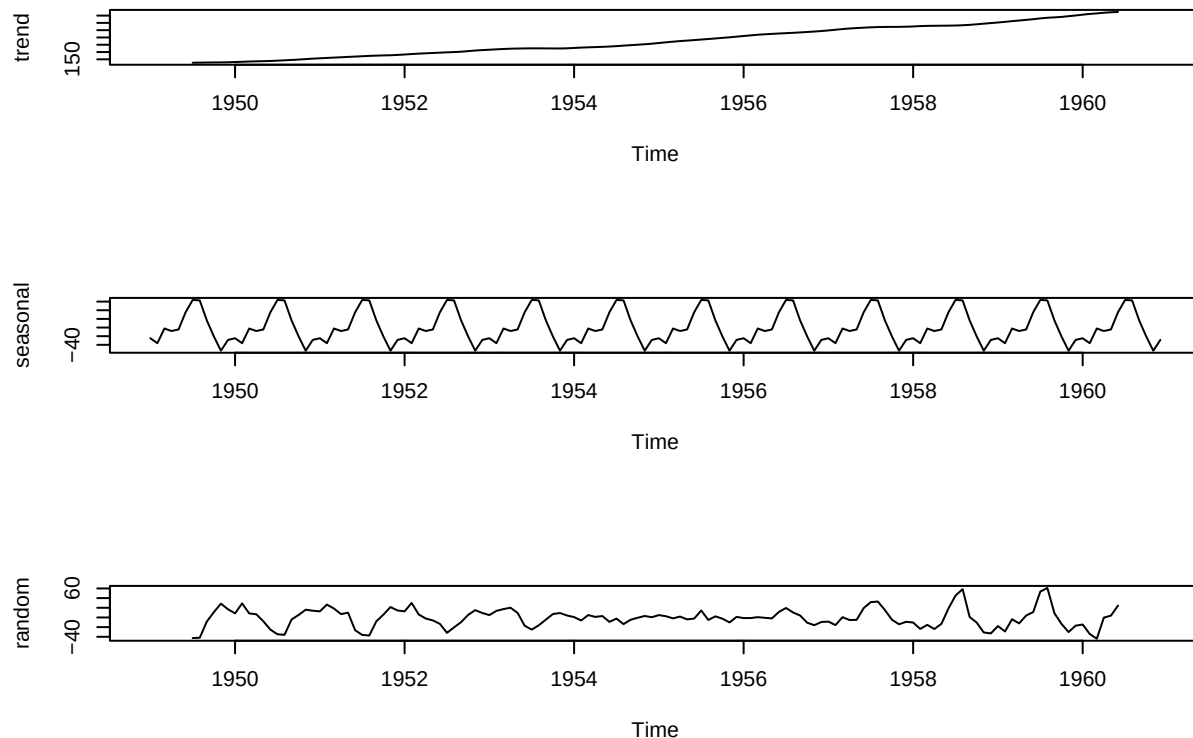
```
dec<-decompose(xt, type="additive")
plot(dec)
```

Decomposition of additive time series



```
trend<-dec$trend
seasonal<-dec$seasonal
random<-dec$random

layout(1:3)
plot(trend)
plot(seasonal)
plot(random)
```



The *decompose* function allows to easily visualize what are the trend, seasonal and random components of the observed time series - as well as to extract such components in very less steps.

Practical 1B

Simulation of Time Series in R

9. IID and Gaussian white noise

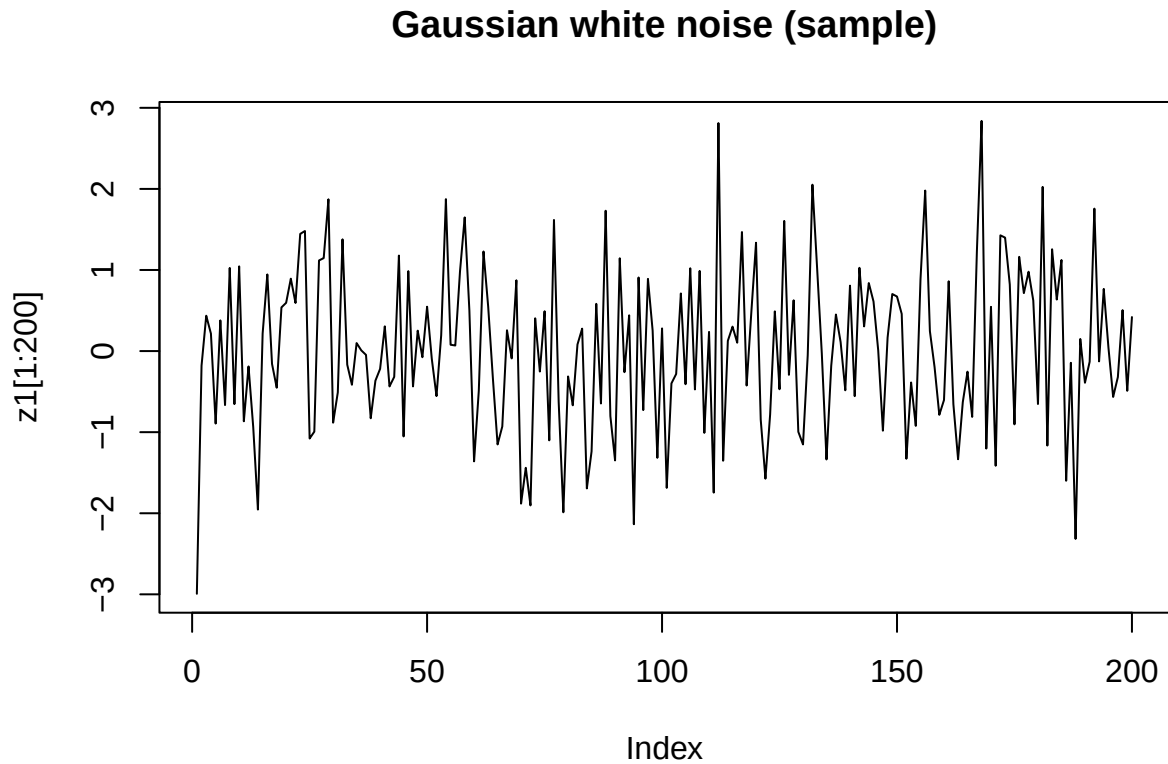
Simulation of **Independent and Identically Distributed (IID) Gaussian White Noise**. First, we define the sample size; we want a sample of 1000 data points. Then, we generate n random values from a standard normal distribution. This is called Gaussian White Noise, meaning: - Values are independent -> no correlation between points - Values are identically distributed -> all come from the same normal distribution

```
# Number of data points
n <- 1000
# Standard white noise
z1 <- rnorm(n)
```

We can visualize the first 200 points as a line plot. As we can see, there's clearly a lack of trend. This is typical for white noise. We can also visualize the distribution with a histogram. It plots the frequency distribution of all 1000 points. Parameter `probability = TRUE` scales the histogram to density, so it can be

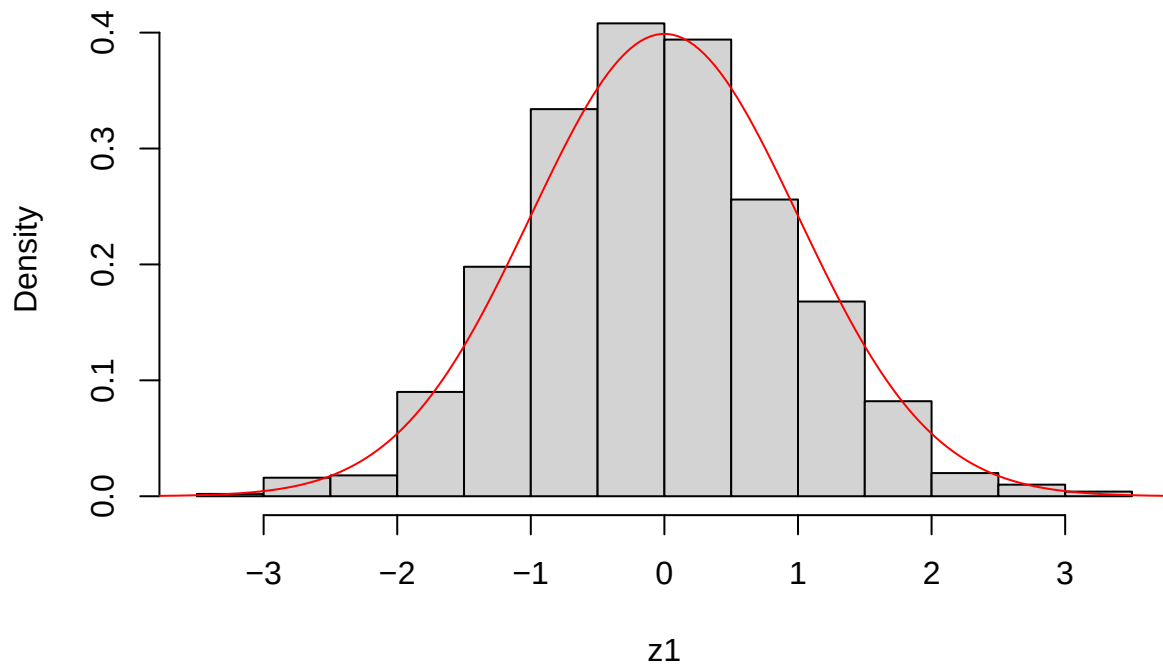
compared with a probability density function. Function `dnorm(x)` computes the standard normal density at points `x`. Function `lines` draw the density as a red line on the histogram. This allows us to compare the sample with the true normal distribution, showing that the sample correctly approximated the theoretical normal distribution.

```
plot(z1[1:200], type = "l", main = "Gaussian white noise (sample)")
```



```
hist(z1, probability = TRUE, main = "Histogram of standard normal sample")
x <- seq(-4, 4, length = 200)
lines(x, dnorm(x), col = "red")
```

Histogram of standard normal sample



Sample definition variations

We can generate the sample without defining first `n`. Although the first approach is better, because it gives us more flexibility when programming.

```
z1 <- rnorm(100)
```

```
z1
```

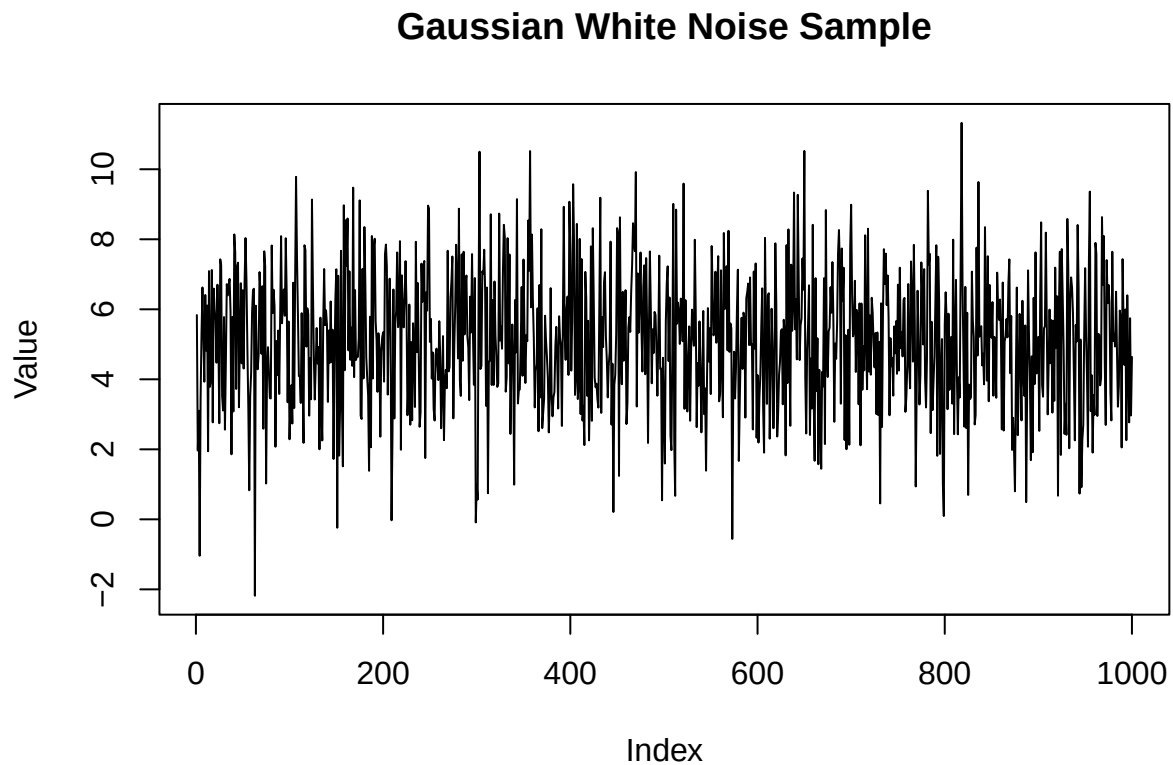
```
## [1] 0.591911059 0.984118173 -1.034357268 2.441706866 0.858068505
## [6] -1.896193165 1.502857008 -0.790786671 -2.362089057 0.659420437
## [11] 0.457454776 -0.419106965 -1.769978425 -1.077772724 -1.359528406
## [16] 0.245597497 1.550507784 0.821882066 0.859014595 -1.014501291
## [21] -1.099487107 -0.146776646 -1.796384888 -1.690710536 -1.499247181
## [26] 1.365613588 0.084038965 -1.172108320 0.314323663 -1.151190721
## [31] 0.561741737 -1.539307321 -1.420350249 -1.072974888 0.968079577
## [36] -0.394664247 -0.072560422 0.519623720 0.697729315 0.021404692
## [41] 0.657520056 0.187158206 0.396153702 1.277864560 -0.607732113
## [46] 1.043287227 0.017971626 0.821494693 0.401781720 -0.451710166
## [51] 0.971050081 -0.968333331 -0.606555837 -0.291248853 -1.383560711
## [56] 0.486959706 -1.350768543 2.077761687 0.494250058 -0.649571185
## [61] 1.058324144 0.641005173 -0.562161170 1.289860202 1.300797830
## [66] -0.513632576 0.208105538 -0.289942128 0.595208614 -0.324790556
## [71] -0.814396979 0.487961267 -0.590716747 -0.459654871 0.883866468
## [76] -0.523829760 -1.467530868 0.901524267 -0.075775109 0.672148320
## [81] 0.920992017 -0.128743743 -0.869501779 1.218519626 1.119246768
## [86] 1.421581649 -1.103988824 0.109980193 -0.035988514 -0.137450152
## [91] -0.612652413 0.740540767 -0.053194700 -0.811664575 0.196339470
```

```
## [96] -0.654226754 -0.003275683 1.105852295 -0.685201192 -0.011272025
```

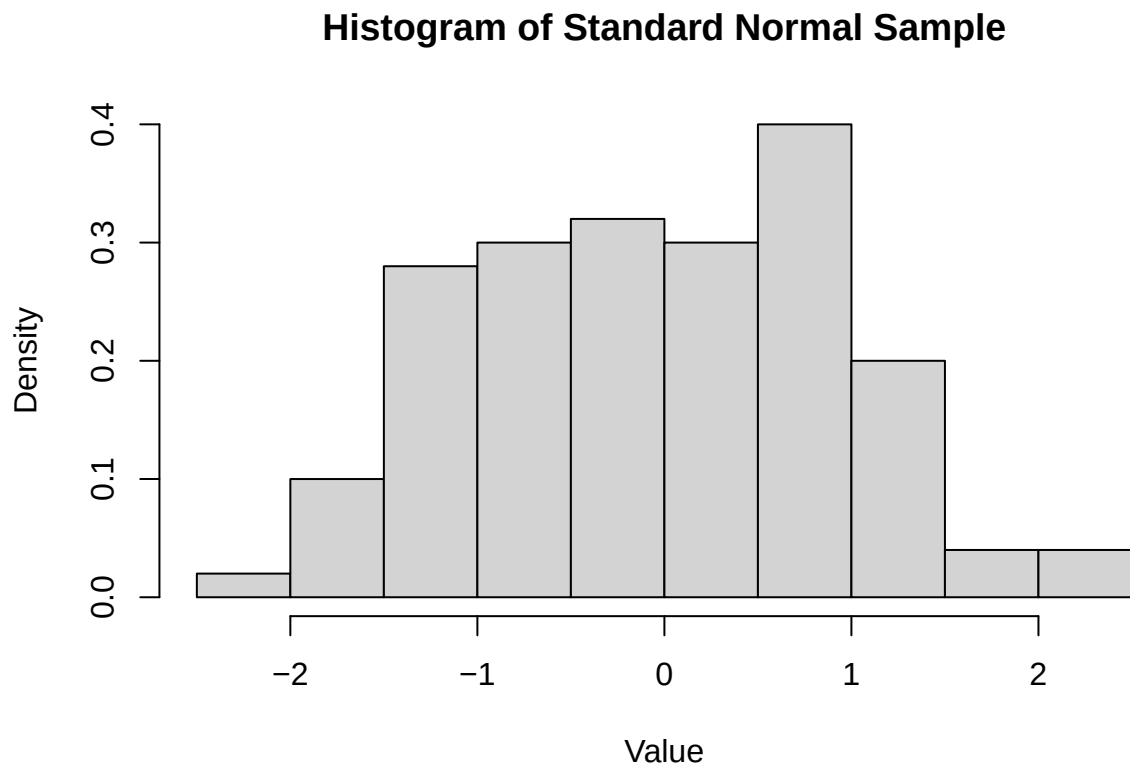
Gaussian White Noise with mean and standard deviation

We can also define a specific mean (μ) and standard deviation (σ). Visualization of the data with a line plot and a histogram.

```
# Gaussian white noise with mean 5 and std 2
mu <- 5
sigma <- 2
# Sample of a normal random variable with mean 5 and std 2
z2 <- rnorm(n, mean = mu, sd = sigma)
plot(z2, type = "l", xlab = "Index", ylab = "Value",
     main = "Gaussian White Noise Sample")
```



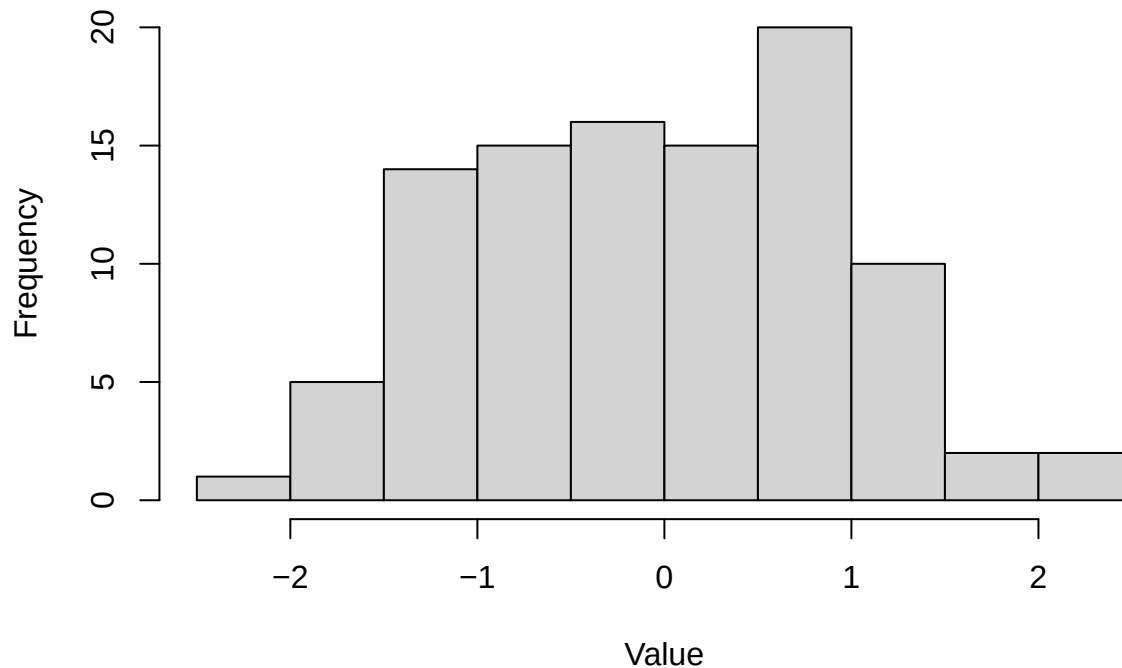
```
hist(z1, freq = FALSE, main = "Histogram of Standard Normal Sample", xlab = "Value")
```



What happens if we remove the parameter `freq=F`? This parameter scales the histogram to a probability density. Removing the parameter plots frequencies instead of densities.

```
hist(z1, main = "Histogram of Standard Normal Sample", xlab = "Value")
```

Histogram of Standard Normal Sample



Changing the sample size (n), mean (mu) and standard deviation (sigma):

```
# Define sets of parameters to explore
n_values <- c(50, 500, 5000)          # sample sizes
mu_values <- c(0, 5)                  # means
sigma_values <- c(1, 3)                # standard deviations

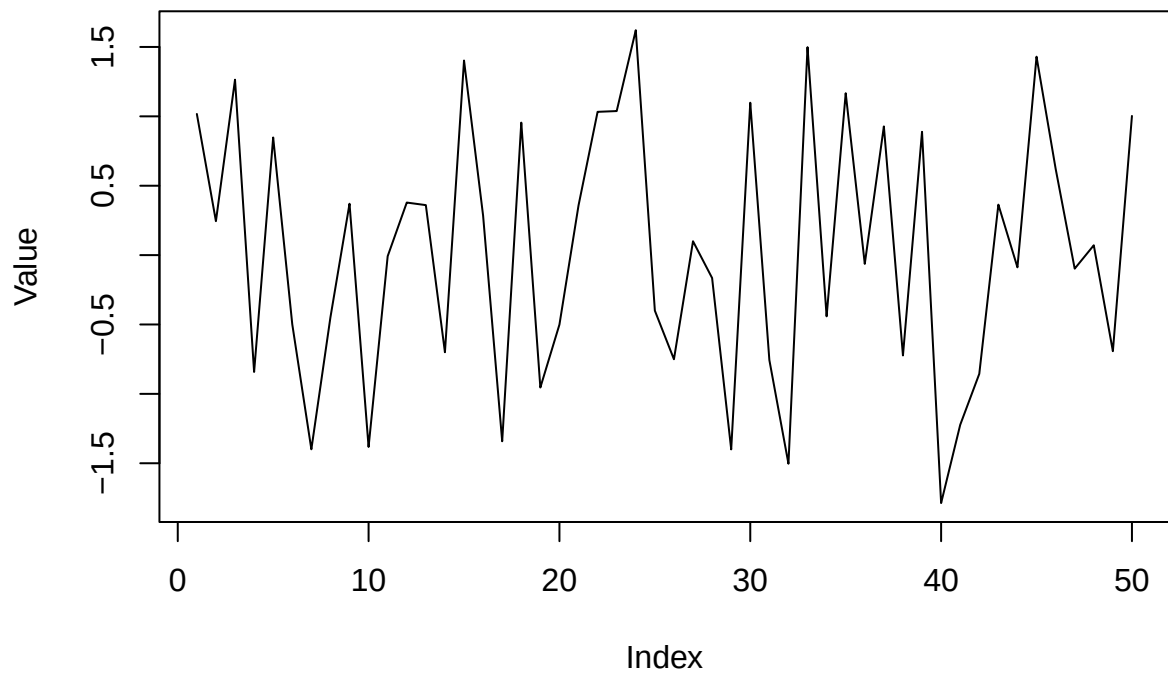
# Loop over all combinations
for(n in n_values){
  for(mu in mu_values){
    for(sigma in sigma_values){
      # Generate Gaussian white noise
      z <- rnorm(n, mean = mu, sd = sigma)

      # Line plot of first 200 points (or full sample if small n)
      plot_len <- min(200, n)
      plot(z[1:plot_len], type="l",
           main=paste0("Line Plot: n=", n, ", mu=", mu, ", sigma=", sigma),
           xlab="Index", ylab="Value")

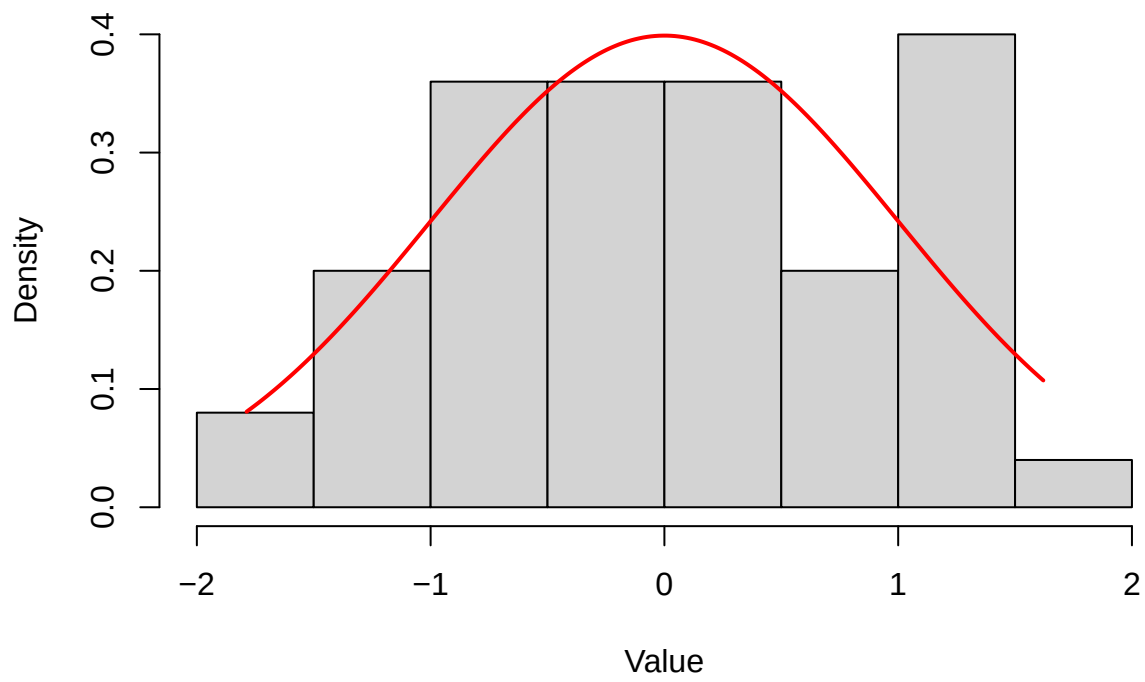
      # Histogram with theoretical density
      hist(z, probability=TRUE,
           main=paste0("Histogram: n=", n, ", mu=", mu, ", sigma=", sigma),
           xlab="Value")
      x <- seq(min(z), max(z), length=200)
      lines(x, dnorm(x, mean=mu, sd=sigma), col="red", lwd=2)
    }
  }
}
```

```
}  
}  
}
```

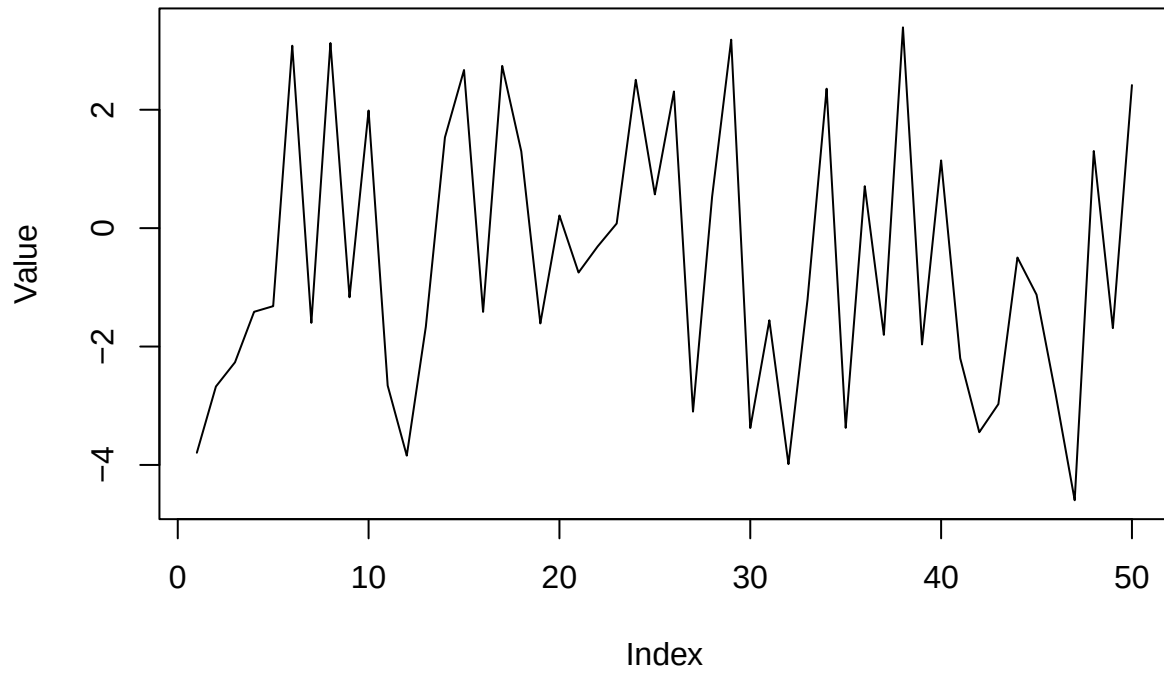
Line Plot: $n=50$, $\mu=0$, $\sigma=1$



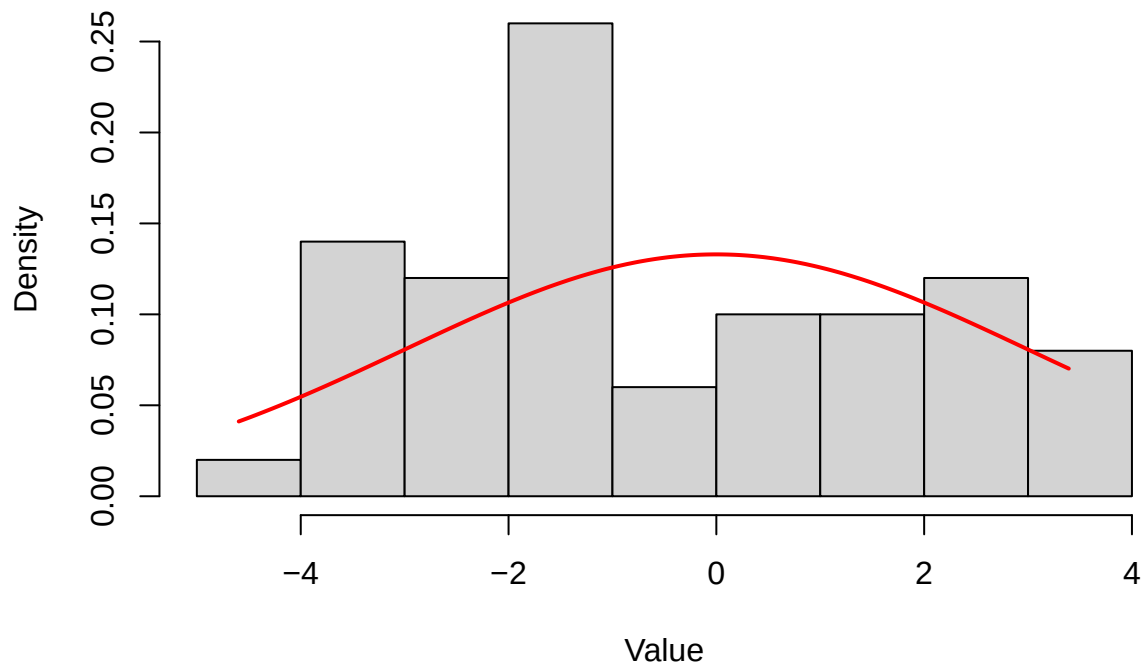
Histogram: $n=50$, $\mu=0$, $\sigma=1$



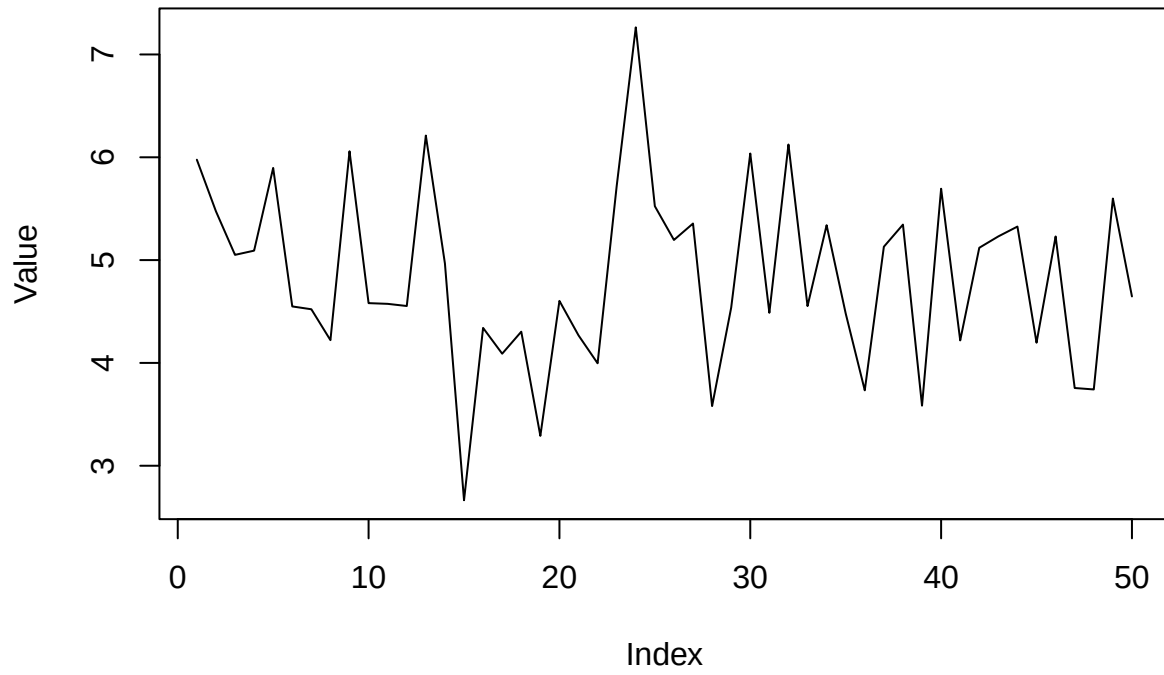
Line Plot: $n=50$, $\mu=0$, $\sigma=3$



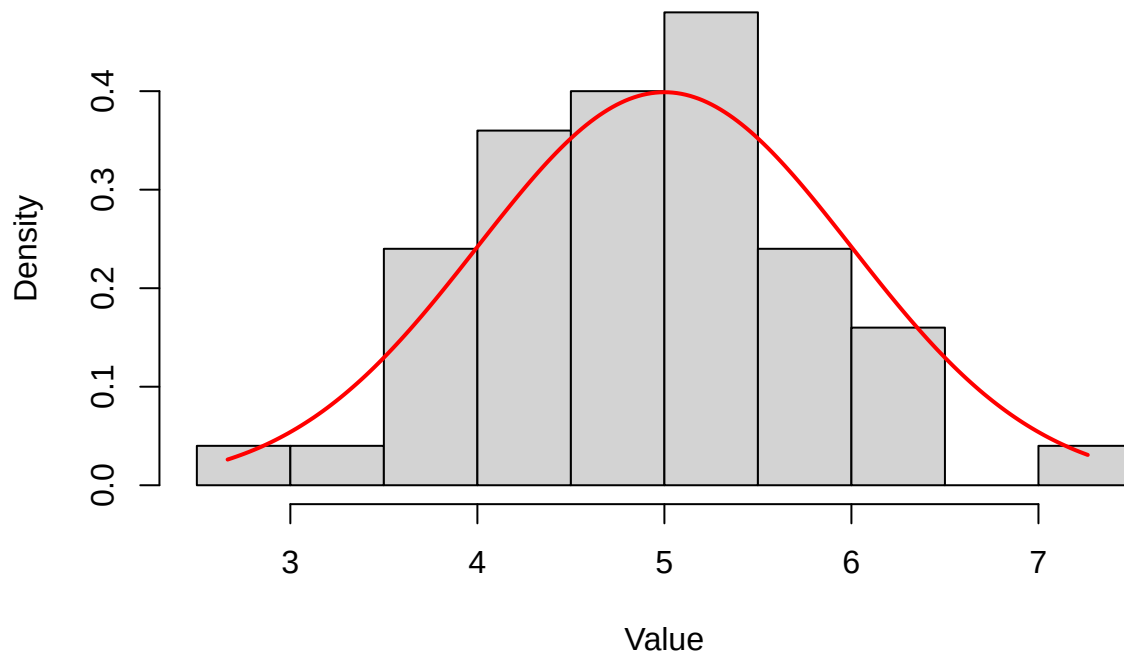
Histogram: $n=50$, $\mu=0$, $\sigma=3$



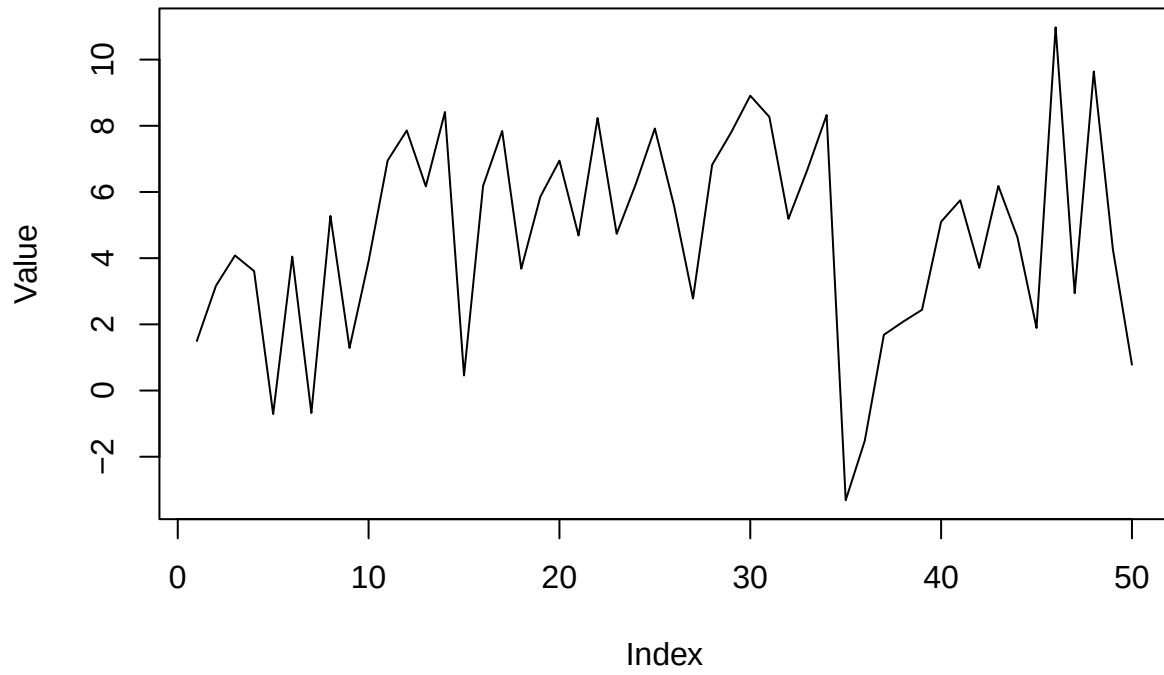
Line Plot: $n=50$, $\mu=5$, $\sigma=1$



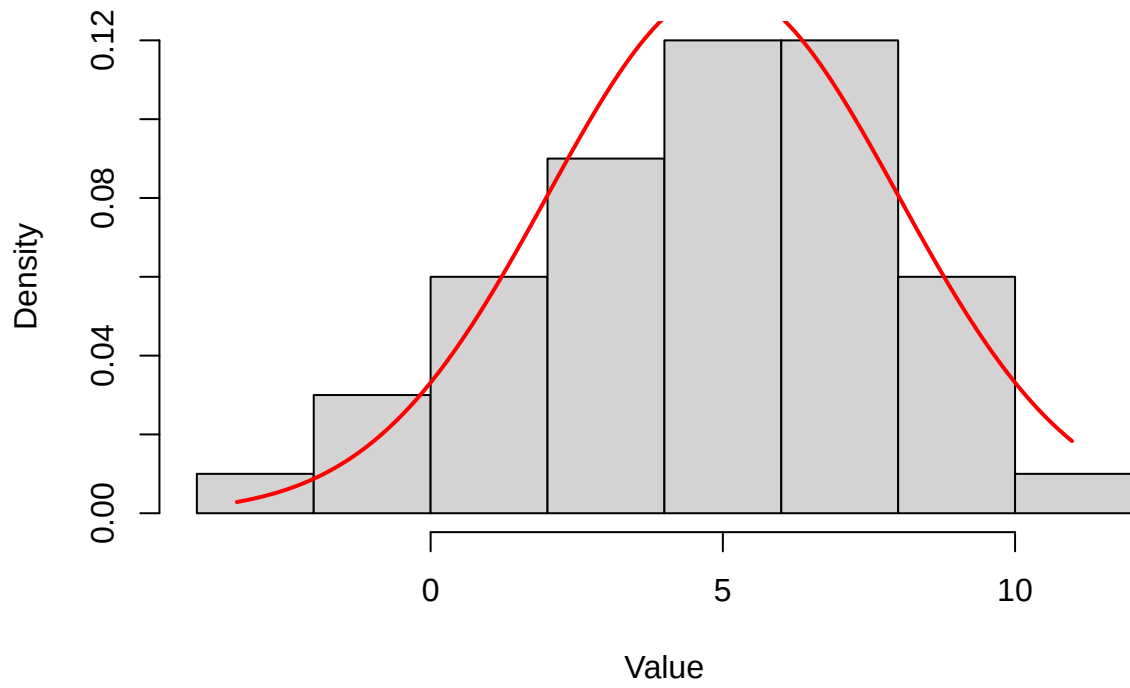
Histogram: $n=50$, $\mu=5$, $\sigma=1$



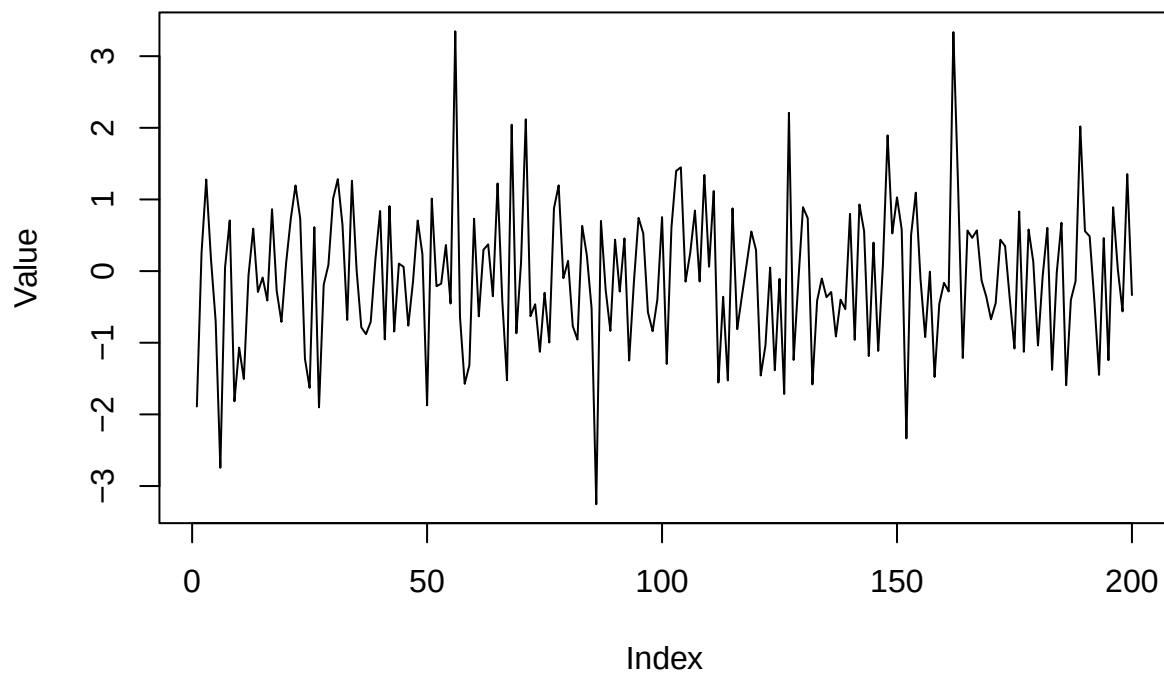
Line Plot: $n=50$, $\mu=5$, $\sigma=3$



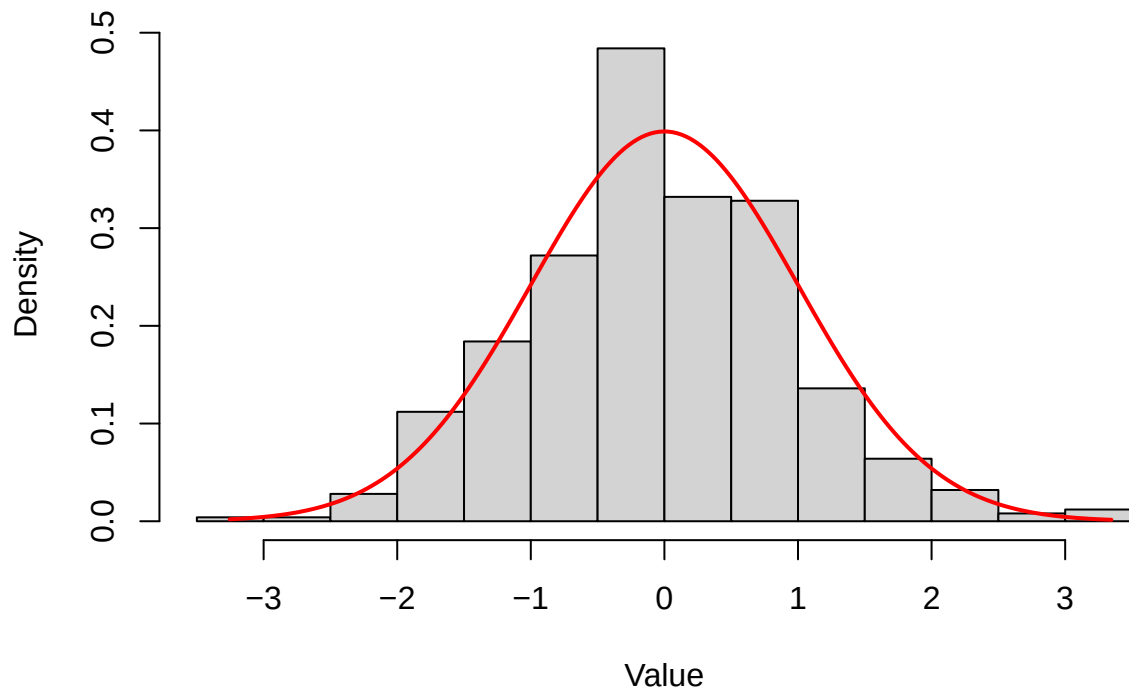
Histogram: $n=50$, $\mu=5$, $\sigma=3$



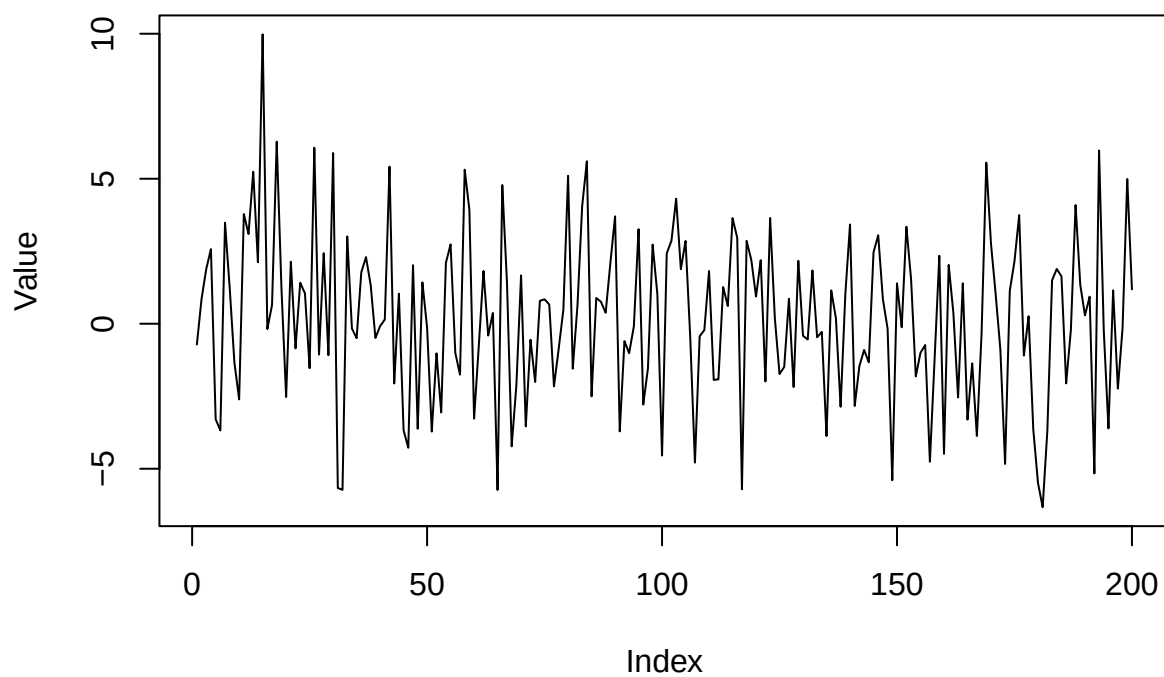
Line Plot: $n=500$, $\mu=0$, $\sigma=1$



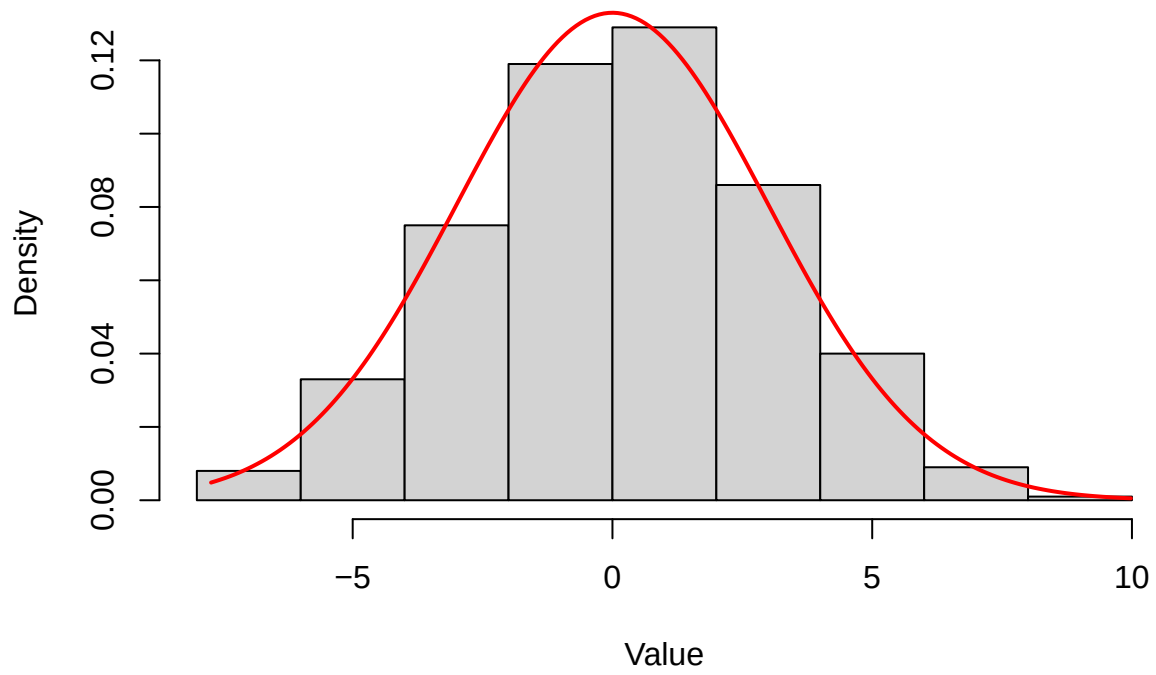
Histogram: n=500, mu=0, sigma=1



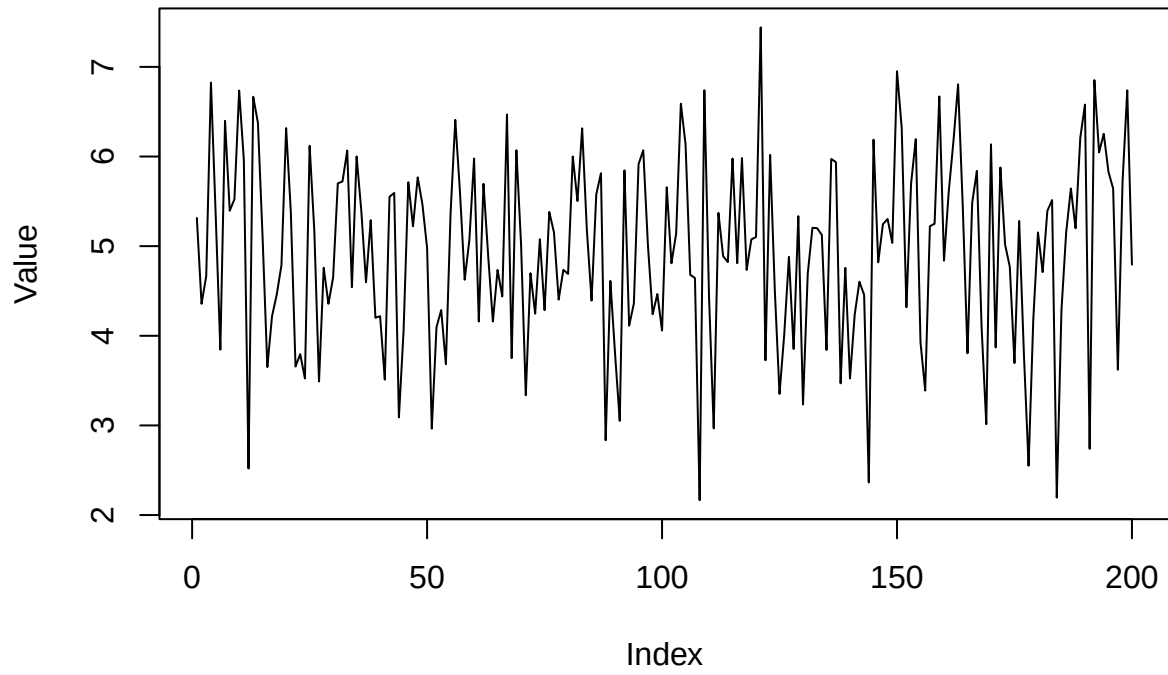
Line Plot: $n=500$, $\mu=0$, $\sigma=3$



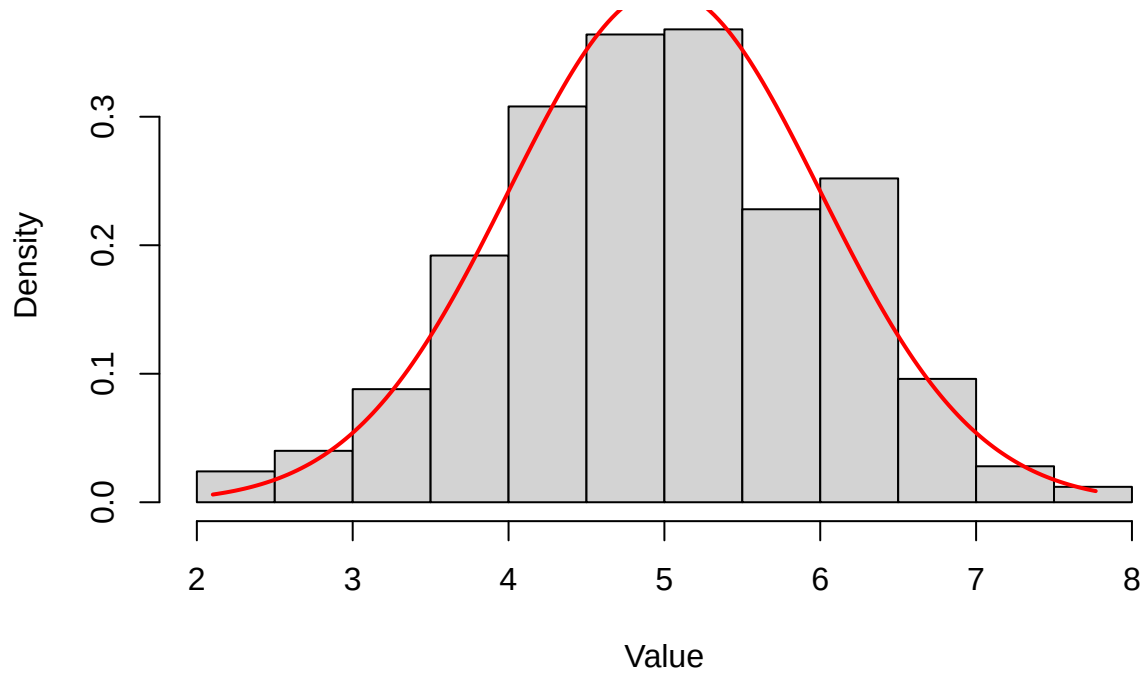
Histogram: n=500, mu=0, sigma=3



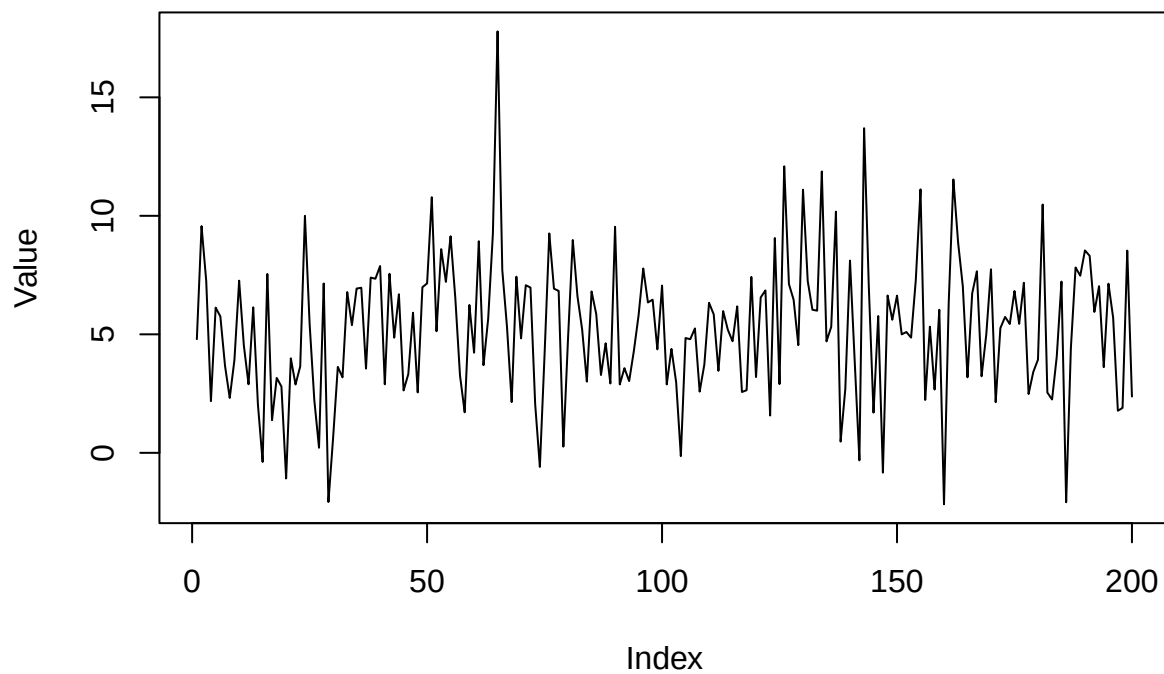
Line Plot: $n=500$, $\mu=5$, $\sigma=1$



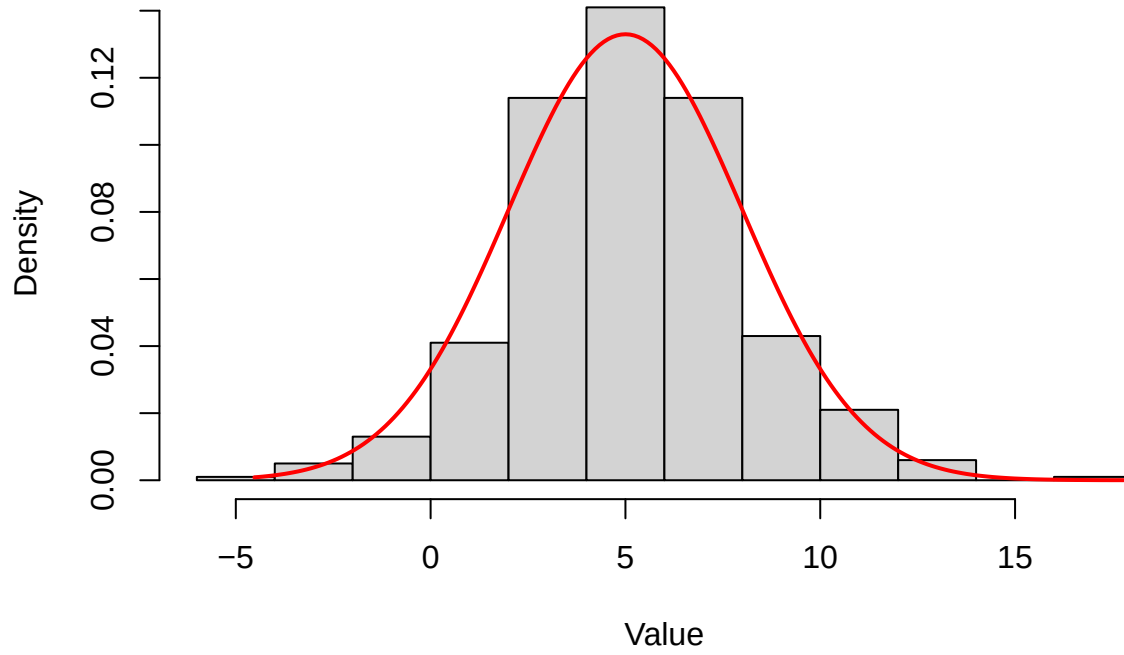
Histogram: $n=500$, $\mu=5$, $\sigma=1$



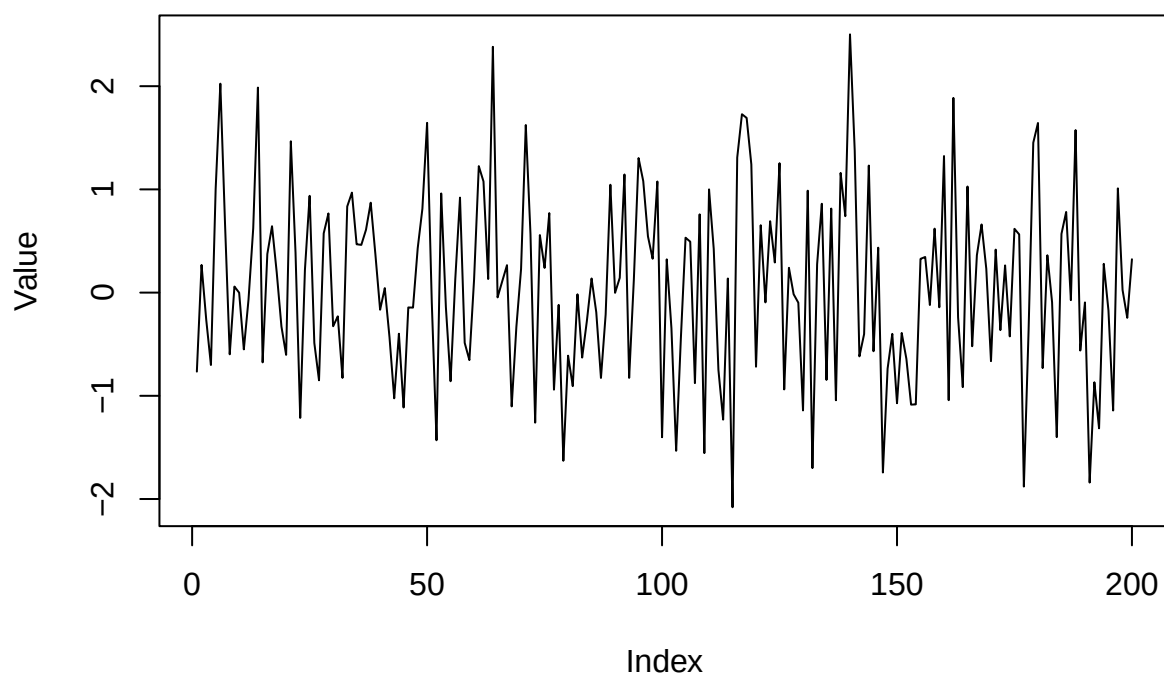
Line Plot: $n=500$, $\mu=5$, $\sigma=3$



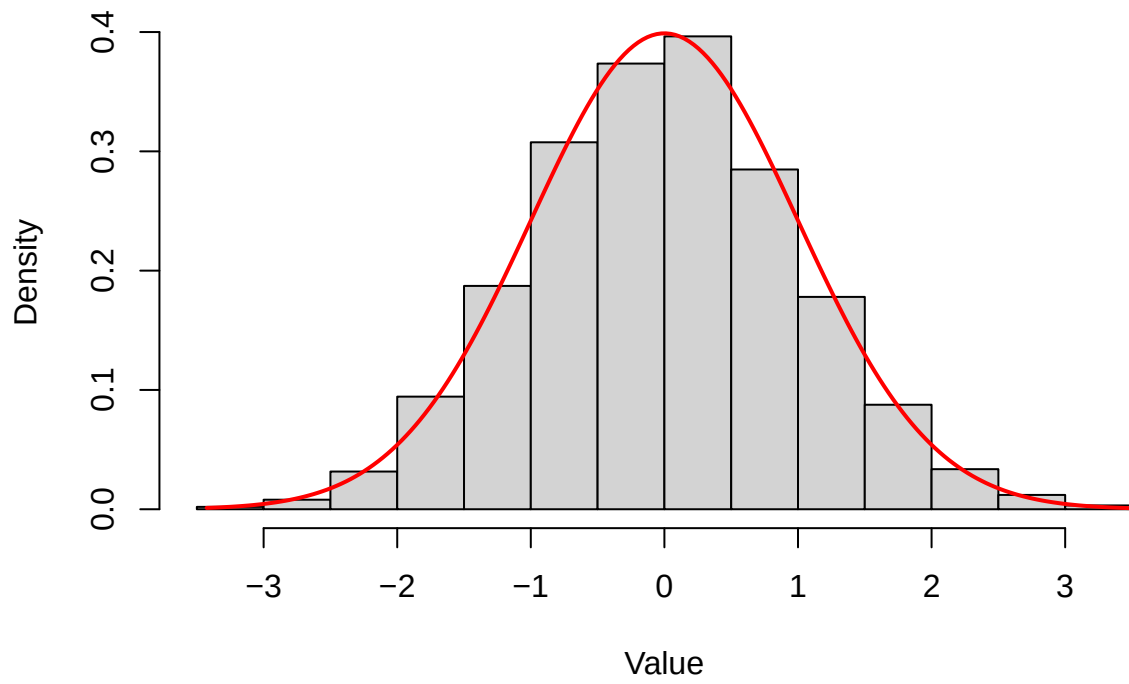
Histogram: n=500, mu=5, sigma=3



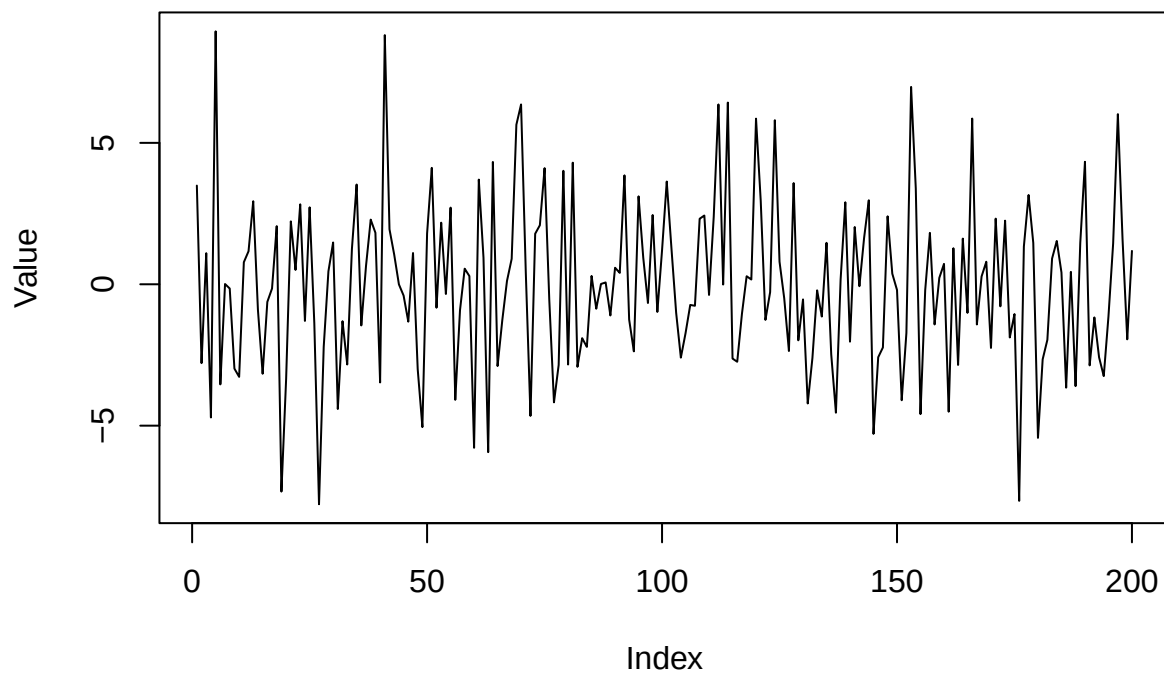
Line Plot: $n=5000$, $\mu=0$, $\sigma=1$



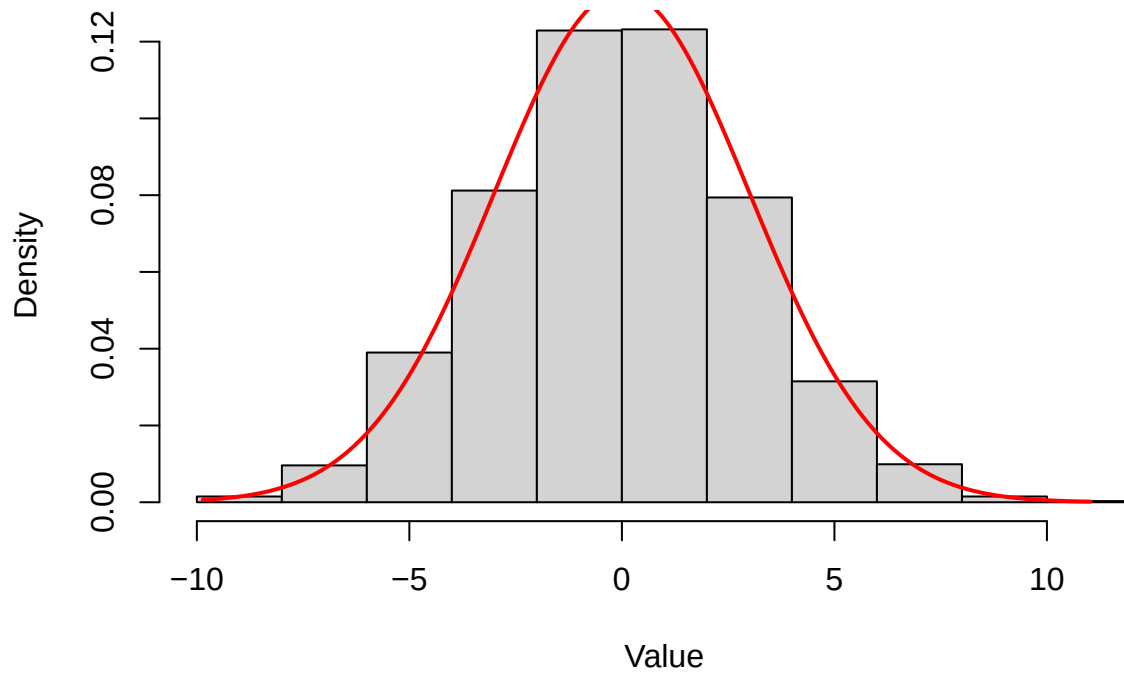
Histogram: $n=5000$, $\mu=0$, $\sigma=1$



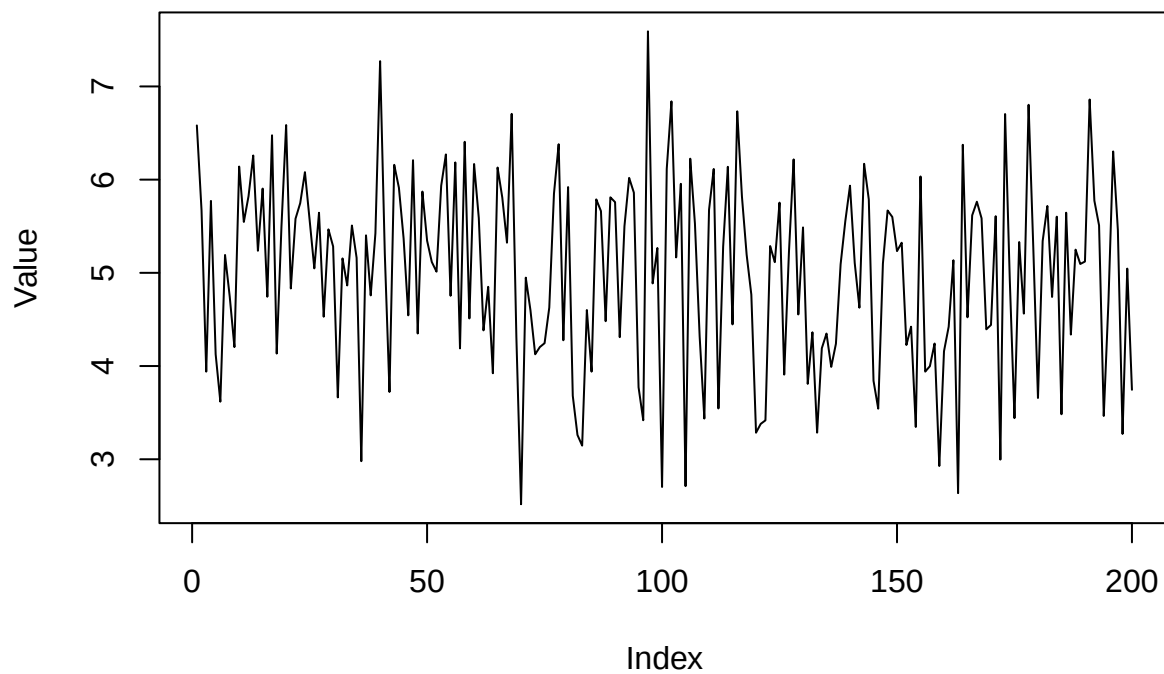
Line Plot: $n=5000$, $\mu=0$, $\sigma=3$



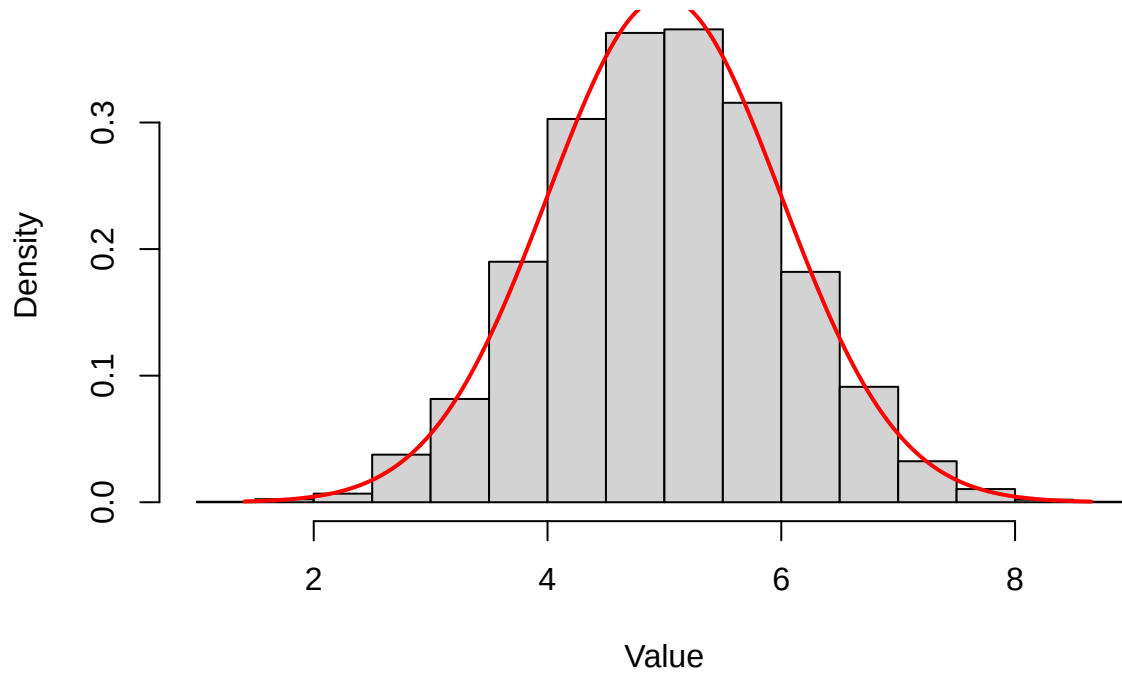
Histogram: n=5000, mu=0, sigma=3



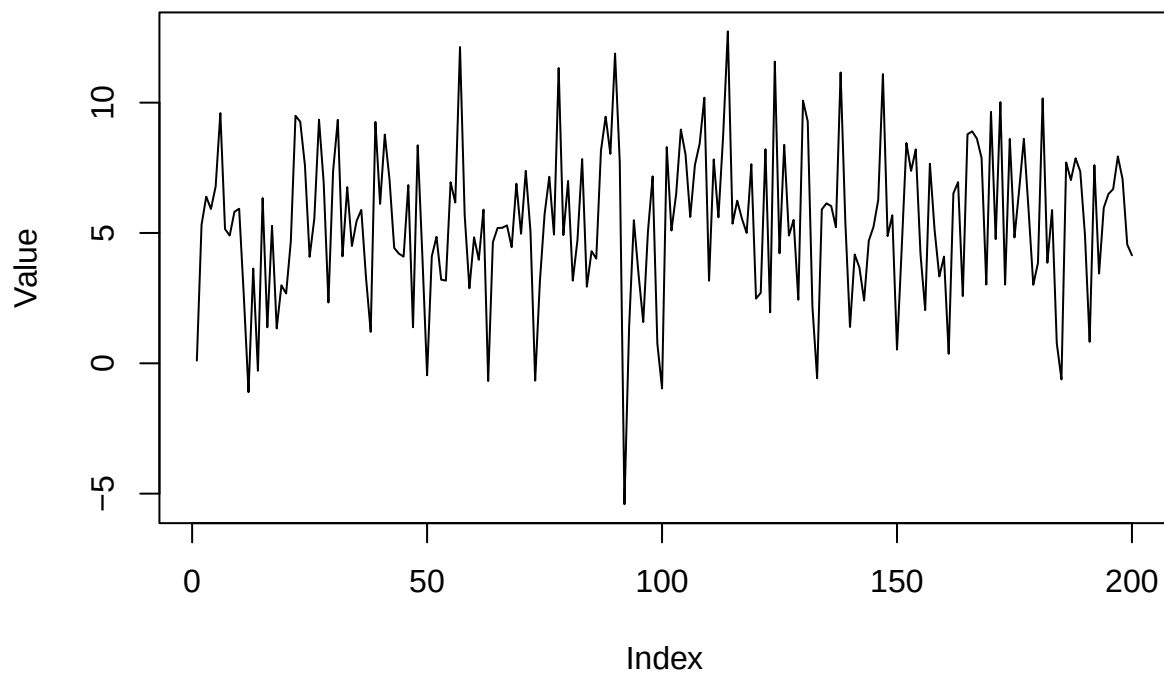
Line Plot: $n=5000$, $\mu=5$, $\sigma=1$



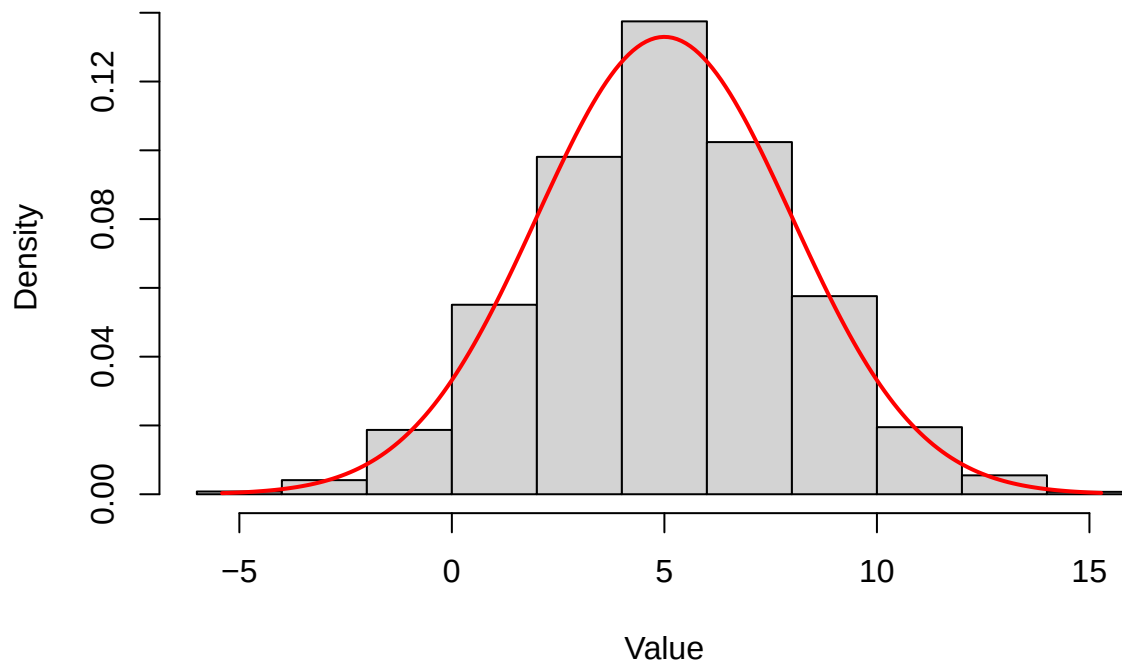
Histogram: $n=5000$, $\mu=5$, $\sigma=1$



Line Plot: $n=5000$, $\mu=5$, $\sigma=3$



Histogram: n=5000, mu=5, sigma=3



- 1. Effect of sample size (n):** - Small: histogram looks noisy, deviates from the red theoretical density line. Line plot shows more spikes, more irregular. - High: histogram matches more closely the theoretical normal distribution, line plot still sows random pattern.
- 2. Effect of mean (mu):** - Changing the mean shifts the center of the histogram and line plot. If $\mu \rightarrow 0$, centered around 0. Overall shape and spread is the same.
- 3. Effects of Standard Deviation (sigma):** - Increasing sigma spreads the data: values more dispersed in the line plot, and histogram becomes wider and flatter. - Decreasing sigma concentrates the data around the mean: narrower and taller histogram.

Random seed

The reproduce the same random sample every time, we can set a seed. By default R uses a random seed.

```
m <- 1
set.seed(m)
w <- rnorm(1000)
w
```

```
## [1] -0.626453811 0.183643324 -0.835628612 1.595280802 0.329507772
## [6] -0.820468384 0.487429052 0.738324705 0.575781352 -0.305388387
## [11] 1.511781168 0.389843236 -0.621240581 -2.214699887 1.124930918
## [16] -0.044933609 -0.016190263 0.943836211 0.821221195 0.593901321
## [21] 0.918977372 0.782136301 0.074564983 -1.989351696 0.619825748
## [26] -0.056128740 -0.155795507 -1.470752384 -0.478150055 0.417941560
```

```

## [31] 1.358679552 -0.102787727 0.387671612 -0.053805041 -1.377059557
## [36] -0.414994563 -0.394289954 -0.059313397 1.100025372 0.763175748
## [41] -0.164523596 -0.253361680 0.696963375 0.556663199 -0.688755695
## [46] -0.707495157 0.364581962 0.768532925 -0.112346212 0.881107726
## [51] 0.398105880 -0.612026393 0.341119691 -1.129363096 1.433023702
## [56] 1.980399899 -0.367221476 -1.044134626 0.569719627 -0.135054604
## [61] 2.401617761 -0.039240003 0.689739362 0.028002159 -0.743273209
## [66] 0.188792300 -1.804958629 1.465554862 0.153253338 2.172611670
## [71] 0.475509529 -0.709946431 0.610726353 -0.934097632 -1.253633400
## [76] 0.291446236 -0.443291873 0.001105352 0.074341324 -0.589520946
## [81] -0.568668733 -0.135178615 1.178086997 -1.523566800 0.593946188
## [86] 0.332950371 1.063099837 -0.304183924 0.370018810 0.267098791
## [91] -0.542520031 1.207867806 1.160402616 0.700213650 1.586833455
## [96] 0.558486426 -1.276592208 -0.573265414 -1.224612615 -0.473400636
## [101] -0.620366677 0.042115873 -0.910921649 0.158028772 -0.654584644
## [106] 1.767287269 0.716707476 0.910174229 0.384185358 1.682176081
## [111] -0.635736454 -0.461644730 1.432282239 -0.650696353 -0.207380744
## [116] -0.392807929 -0.319992869 -0.279113303 0.494188331 -0.177330482
## [121] -0.505957462 1.343038825 -0.214579409 -0.179556530 -0.100190741
## [126] 0.712666307 -0.073564404 -0.037634171 -0.681660479 -0.324270272
## [131] 0.060160440 -0.588894486 0.531496193 -1.518394082 0.306557861
## [136] -1.536449824 -0.300976127 -0.528279904 -0.652094781 -0.056896778
## [141] -1.914359426 1.176583312 -1.664972436 -0.463530401 -1.115920105
## [146] -0.750819001 2.087166546 0.017395620 -1.286300530 -1.640605534
## [151] 0.450187101 -0.018559833 -0.318068375 -0.929362147 -1.487460310
## [156] -1.075192297 1.000028804 -0.621266695 -1.384426847 1.869290622
## [161] 0.425100377 -0.238647101 1.058483049 0.886422651 -0.619243048
## [166] 2.206102465 -0.255027030 -1.424494650 -0.144399602 0.207538339
## [171] 2.307978399 0.105802368 0.456998805 -0.077152935 -0.334000842
## [176] -0.034726028 0.787639606 2.075245009 1.027392439 1.207908398
## [181] -1.231323422 0.983895570 0.219924804 -1.467250029 0.521022743
## [186] -0.158754605 1.464587312 -0.766082000 -0.430211754 -0.926109497
## [191] -0.177103961 0.402011779 -0.731748173 0.830373168 -1.208082786
## [196] -1.047984413 1.441157707 -1.015847465 0.411974712 -0.381076051
## [201] 0.409401840 1.688873286 1.586588433 -0.330907801 -2.285235535
## [206] 2.497661590 0.667066167 0.541327336 -0.013399523 0.510108423
## [211] -0.164375832 0.420694643 -0.400246744 -1.370207878 0.987838267
## [216] 1.519745025 -0.308740569 -1.253289756 0.642241306 -0.044709137
## [221] -1.733218407 0.002131860 -0.630300334 -0.340968580 -1.156572363
## [226] 1.803141908 -0.331132036 -1.605513412 0.197193439 0.263175646
## [231] -0.985826700 -2.888920672 -0.640481703 0.570507636 -0.059723276
## [236] -0.098178744 0.560820729 -1.186458639 1.096777044 -0.005344028
## [241] 0.707310667 1.034107735 0.223480415 -0.878707613 1.162964556
## [246] -2.000164945 -0.544790740 -0.255670709 -0.166121037 1.020463909
## [251] 0.136221893 0.407167603 -0.069654813 -0.247664342 0.695550807
## [256] 1.146228357 -2.403096215 0.572739555 0.374724407 -0.425267722
## [261] 0.951012808 -0.389237182 -0.284330662 0.857409778 1.719627299
## [266] 0.270054901 -0.422184010 -1.189113295 -0.331032979 -0.939829327
## [271] -0.258932583 0.394379168 -0.851857092 2.649166881 0.156011676
## [276] 1.130207267 -2.289123980 0.741001157 -1.316245160 0.919803678
## [281] 0.398130155 -0.407528579 1.324258630 -0.701231669 -0.580614304
## [286] -1.001072181 -0.668178607 0.945184953 0.433702150 1.005159218
## [291] -0.390118664 0.376370292 0.244164924 -1.426257342 1.778429287
## [296] 0.134447661 0.765598999 0.955136677 -0.050565701 -0.305815420

```

```

## [301] 0.893673702 -1.047298149 1.971337386 -0.383632106 1.654145302
## [306] 1.512212694 0.082965734 0.567220915 -1.024548480 0.323006503
## [311] 1.043612458 0.099078487 -0.454136909 -0.655781852 -0.035922423
## [316] 1.069161461 -0.483974930 -0.121010111 -1.294140004 0.494312836
## [321] 1.307901520 1.497041009 0.814702731 -1.869788790 0.482029504
## [326] 0.456135603 -0.353400286 0.170489471 -0.864035954 0.679230774
## [331] -0.327101015 -1.569082185 -0.367450756 1.364434929 -0.334281365
## [336] 0.732750042 0.946585640 0.004398704 -0.352322306 -0.529695509
## [341] 0.739589226 -1.063457415 0.246210844 -0.289499367 -2.264889356
## [346] -1.408850456 0.916019329 -0.191278951 0.803283216 1.887474463
## [351] 1.473881181 0.677268492 0.379962687 -0.192798426 1.577891795
## [356] 0.596234109 -1.173576941 -0.155642535 -1.918909820 -0.195258846
## [361] -2.592327670 1.314002167 -0.635543001 -0.429978839 -0.169318332
## [366] 0.612218174 0.678340177 0.567951972 -0.572542604 -1.363291256
## [371] -0.388722244 0.277914132 -0.823081122 -0.068840934 -1.167662326
## [376] -0.008309014 0.128855402 -0.145875628 -0.163910957 1.763552003
## [381] 0.762586512 1.111431081 -0.923206953 0.164341838 1.154825187
## [386] -0.056521425 -2.129360648 0.344845762 -1.904955446 -0.811170153
## [391] 1.324004321 0.615636849 1.091668956 0.306604862 -0.110158762
## [396] -0.924312773 1.592913754 0.045010598 -0.715128401 0.865223100
## [401] 1.074440958 1.895654774 -0.602997304 -0.390867821 -0.416222032
## [406] -0.375657423 -0.366630946 -0.295677453 1.441820410 -0.697538292
## [411] -0.388167506 0.652536452 1.124772447 -0.772110803 -0.508086216
## [416] 0.523620590 1.017754227 -0.251164588 -1.429993447 1.709121032
## [421] 1.435069572 -0.710371146 -0.065067574 -1.759468735 0.569722972
## [426] 1.612346798 -1.637280647 -0.779568513 -0.641176934 -0.681131394
## [431] -2.033285596 0.500963559 -1.531798140 -0.024997639 0.592984721
## [436] -0.198195421 0.892008392 -0.025715071 -0.647660451 0.646359415
## [441] -0.433832740 1.772611185 -0.018259711 0.852814994 0.205162903
## [446] -3.008048599 -1.366111931 -0.424102260 0.236803664 -2.342723120
## [451] 0.961696633 -0.604425734 -0.752877279 -1.555611593 -1.453893738
## [456] 0.056331836 0.509369407 -2.097882960 -1.004361979 0.535771722
## [461] -0.453037085 2.165368502 1.245746673 0.595498034 0.004884450
## [466] 0.279360782 -0.705906125 0.628017153 1.480213960 1.083429910
## [471] -0.813244257 -1.618876849 -0.109655699 0.440889371 1.350993980
## [476] -1.318609485 0.364384593 0.233499835 1.193955261 -0.027909972
## [481] -0.357298855 -1.146814136 -0.517420484 -0.362123773 2.350554326
## [486] 2.446531376 -0.166703279 -1.043667439 -1.972934934 0.514671633
## [491] -1.090573584 2.284659326 -0.885617573 0.111106430 3.810276681
## [496] -1.108909998 0.307566624 -1.106894472 0.347653649 -0.873264535
## [501] 0.077303123 -0.296868642 -1.183242240 0.011292688 0.991601036
## [506] 1.593967454 -1.372711271 -0.249610933 1.159424527 -1.114222348
## [511] -2.528500689 -0.935902559 -0.967239458 0.047488592 -0.403736793
## [516] 0.231496128 -0.422372408 0.374118395 -0.366005775 1.190101447
## [521] -0.737327525 0.290666645 -0.884849568 0.208006479 -0.047730172
## [526] -1.684520646 -0.144226557 1.180213666 0.681399923 0.143247631
## [531] -1.192316444 1.169228653 0.079201709 -0.451773753 1.642028213
## [536] -0.769592322 0.303360961 1.281737421 0.602222795 -0.307022265
## [541] -0.418418103 0.355135530 0.513481115 0.018607400 1.318448972
## [546] -0.065832000 -0.700296078 0.537326132 -2.201782322 0.391973744
## [551] 0.496960952 -0.224874715 -1.117143165 -0.394994603 1.549830342
## [556] -0.743514480 -2.331712118 0.812245442 -0.501310657 -0.510886566
## [561] -1.215364041 -0.022558628 0.701239300 -0.587482026 -0.606727941
## [566] 1.096640215 -0.247509677 -0.159901713 -0.625778251 0.900434636

```

```

## [571] -0.994193629 0.849250386 0.805702289 -0.467600936 0.848420314
## [576] 0.986769864 0.575620289 2.024842045 -1.962353191 -1.164920931
## [581] -1.376519214 0.167679934 1.584629079 1.677888953 0.488296698
## [586] 0.878673263 -0.144874874 0.468971760 0.376235477 -0.761040275
## [591] -0.293294934 -0.134841264 1.393845816 -1.036988690 -2.114335148
## [596] 0.768278218 -0.816160621 -0.436106923 0.904705031 -0.763086265
## [601] -0.341066980 1.502424534 0.528307712 0.542191355 -0.136673356
## [606] -1.136733853 -1.496627154 -0.223385644 2.001719228 0.221703816
## [611] 0.164372909 0.332623609 -0.385207999 -1.398754027 2.675740796
## [616] -0.423686089 -0.298601512 -1.792341727 -0.248008225 -0.247303918
## [621] -0.255510379 -1.786938100 1.784662816 1.763586348 0.689600222
## [626] -1.100740644 0.714509357 -0.246470317 -0.319786166 1.362644293
## [631] -1.227882590 -0.511219233 -0.731194999 0.019752007 -1.572863915
## [636] -0.703333270 0.715932089 0.465214906 -0.973902306 0.559217730
## [641] -2.432639745 -0.340484927 0.713033195 -0.659037386 -0.036402623
## [646] -1.593286302 0.847792797 -1.850388849 -0.323650632 -0.255248113
## [651] 0.060921227 -0.823491629 1.829730485 -1.429916216 0.254137143
## [656] -2.939773695 0.002415809 0.509665571 -1.084720001 0.704832977
## [661] 0.330976350 0.976327473 -0.843339880 -0.970579905 -1.771531349
## [666] -0.322470342 -1.338800742 0.688156028 0.071280652 2.189752359
## [671] -1.157707599 1.181688064 -0.527368362 -1.456628011 0.572967370
## [676] -1.433377705 -1.055185019 -0.733111877 0.210907264 -0.998920727
## [681] 1.077850323 -1.198974383 0.216637035 0.143087030 -1.065750091
## [686] -0.428623411 -0.656179477 0.959394327 1.556052636 -1.040796434
## [691] 0.930572409 -0.075445931 -1.967195349 -0.755903643 0.461149161
## [696] 0.145106631 -2.442311321 0.580318685 0.655051998 -0.304508837
## [701] -0.707568233 1.971572014 -0.089998681 -0.014017252 -1.123456937
## [706] -1.344130123 -1.523155771 -0.421968210 1.360924464 1.753794845
## [711] 1.568364734 1.296755570 -0.237596251 -1.224150136 -0.327812680
## [716] -2.412450276 -0.313792873 1.659878705 0.130953101 1.095888683
## [721] 0.489340957 -0.778910295 1.743559353 -0.078387285 -0.975553793
## [726] 0.070659825 -1.518599529 0.863779033 0.501568385 -0.354781330
## [731] -0.488428888 0.936293947 -1.062408386 -0.983820872 0.424247877
## [736] -0.451313481 0.925084797 -0.198620810 1.194851017 0.495544705
## [741] -2.245152574 -1.335371360 1.282775206 0.690795904 -0.967062668
## [746] -1.345793685 1.033665389 -0.811776460 1.801725479 1.771541960
## [751] -1.454691370 -0.845654313 -1.250479659 0.667288068 -1.290769688
## [756] -2.035003538 2.021346994 1.005973488 0.817123601 -0.663988284
## [761] -0.011281231 0.619677256 -1.281238742 -0.124261326 0.175741655
## [766] 1.692773794 0.642132717 1.282233032 0.140546974 -1.112502683
## [771] -0.339676972 -1.664764639 0.928851762 1.416826835 -0.062720778
## [776] -0.980902336 1.087150251 0.139327027 -0.386272098 1.123585423
## [781] -0.759845657 1.148959103 -0.842476259 0.391413340 0.891377242
## [786] -1.335258713 0.398123478 -0.111586803 0.675743917 -0.788597935
## [791] -0.086986334 1.382284008 0.168490164 0.823190948 -0.220894598
## [796] -1.029391655 -0.010925691 -1.224991155 -2.596111388 1.169122592
## [801] -1.086908817 -1.826083013 0.995281807 -0.011861781 -0.599628395
## [806] -0.177947987 -0.425981342 0.996658776 0.727660709 -1.726630596
## [811] 0.353398496 0.726813666 0.668260976 -2.424317309 -0.235357425
## [816] 1.979633321 0.796794539 -1.709276181 -1.663668712 0.491109552
## [821] -0.174055486 0.961290564 0.293826662 0.080999364 0.183661843
## [826] 0.166255035 -1.269599066 2.349493321 -1.412005407 -0.016961493
## [831] -0.544319353 1.800112333 1.011440176 -0.563716556 0.205420795
## [836] 1.165461950 2.236322840 0.302265076 -1.042506602 -0.983542313

```

```
## [841] 2.005718580 -2.070571484 3.055742369 -0.261350594 -0.454393259
## [846] 0.157560555 0.933388728 0.302828276 -1.956150222 0.353536709
## [851] 0.450424514 0.659550871 -1.031420728 -2.371022881 -0.324576308
## [856] -0.944298751 -0.765889998 -0.953779267 -0.398004450 -0.311217063
## [861] 0.796092713 0.986428344 -0.794531662 -0.308817972 0.361444766
## [866] 1.398791105 -0.056070420 -1.698873492 0.231852545 -0.119090671
## [871] 1.772492853 0.343422165 -0.623049782 -0.439522294 -0.505296790
## [876] 0.186035137 0.176417798 0.915848207 0.320176726 -0.366687297
## [881] -0.940611295 0.634702931 -0.062488482 0.182837868 1.103641018
## [886] 1.752035619 -0.953816460 1.644080457 -0.866733529 0.266352192
## [891] 0.222370495 -0.276908510 1.394253041 -0.658912000 0.660531281
## [896] -0.013255700 -0.931480299 1.214689143 -2.088340366 -0.526152469
## [901] -1.541402567 0.194321070 0.264422549 -1.118735169 0.650952956
## [906] -1.032900239 0.659201499 0.237829400 0.715275947 -0.938483039
## [911] 0.095354010 -0.462819420 -1.468882156 0.152686506 1.773762611
## [916] -0.648070934 -0.199817476 0.689243733 0.036145510 1.943536312
## [921] 0.737213734 2.321333933 0.348909346 -1.133916661 0.421335268
## [926] -0.924556257 -1.007062366 -0.189474330 0.933916700 0.343910000
## [931] 0.814020276 0.915341002 -0.171852130 -2.402231114 0.795906852
## [936] 2.169115869 0.058383499 -1.354914450 -0.367550766 -0.934517579
## [941] -0.041639220 0.676112006 0.866436147 0.235175025 -0.933970133
## [946] 0.813252167 1.348318555 2.251882776 -0.493652434 0.474092599
## [951] 1.193691666 -0.116395472 0.524858556 0.214421481 -0.134449535
## [956] 0.168509103 0.964732770 0.408778914 -0.466438215 -2.239782976
## [961] -0.795063185 -0.019955117 -2.514425122 2.210952032 -1.488762230
## [966] -1.160751876 1.457738225 -2.189875949 0.739062681 -0.344008769
## [971] 0.455876742 0.878110263 -0.958463923 -0.705844846 -2.996949303
## [976] -0.961052324 0.380188254 0.505067590 2.027056004 0.064627029
## [981] 0.463658648 0.074778333 -0.486836236 0.748910821 0.464234581
## [986] 0.129420458 -0.815460652 -0.040150452 0.781416978 0.676452747
## [991] -0.482652526 -0.669113468 0.512801322 1.048909929 0.121058163
## [996] -0.313292885 -0.880670730 -0.419286891 -1.482751678 -0.697318199
```

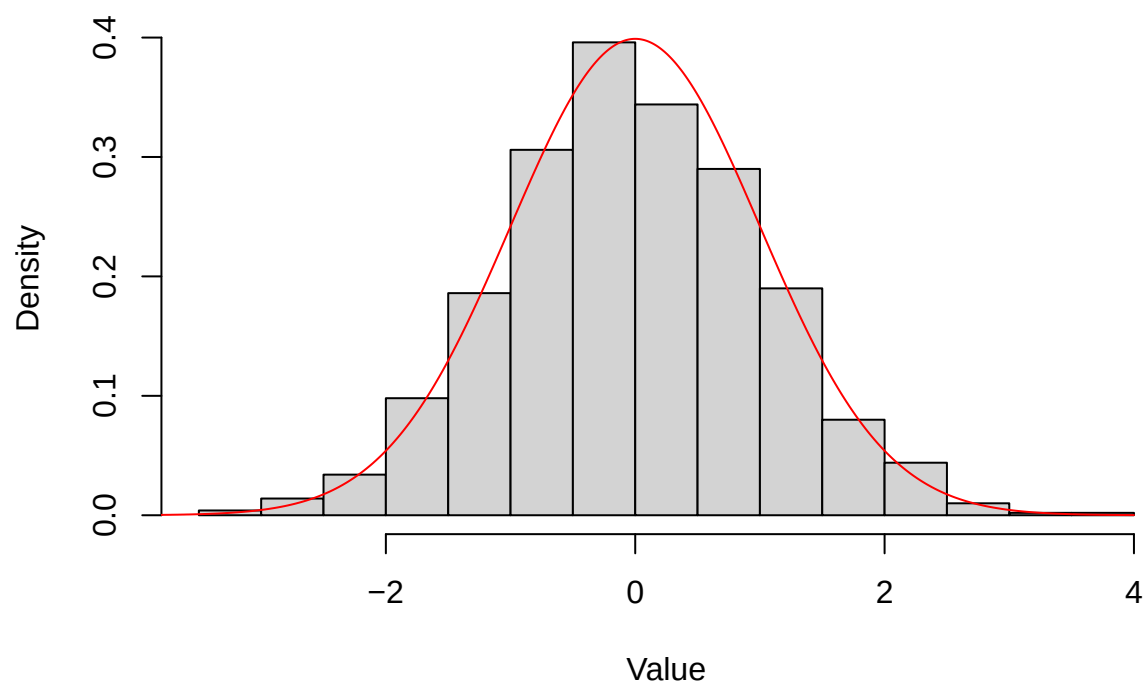
Comparison of samples from the underlying population

Overlay the theoretical normal density on the histogram:

```
n <- 1000
x <- seq(-4, 4, length = 1000)

hist(rnorm(n), prob = TRUE, main = "Sample vs Theoretical Density", xlab = "Value")
points(x, dnorm(x), type = "l", col = "red")
```

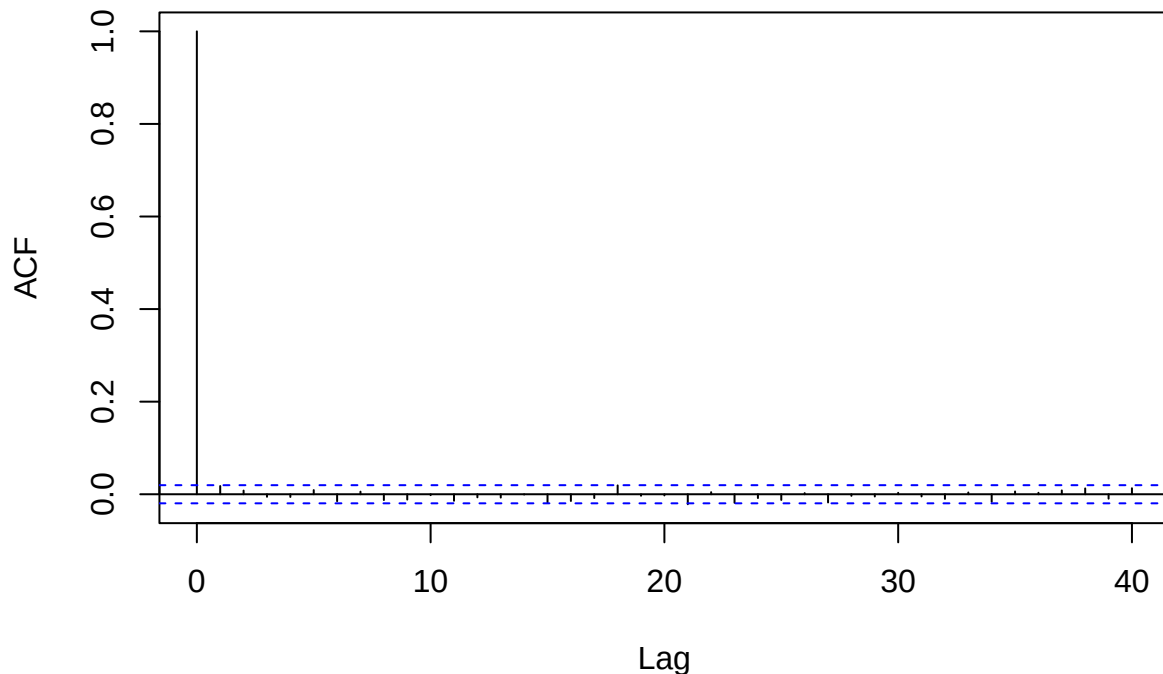
Sample vs Theoretical Density



And autocorrelation function (ACF) for IID Gaussian noise:

```
N <- 10000
mu <- 5
sigma <- 2
z <- rnorm(N, mean = mu, sd = sigma)
acf(z, main = "Correlogram of IID Gaussian Noise")
```

Correlogram of IID Gaussian Noise



As we are working with IID noise, the ACF shows no significant correlation -> independence between observations.

10. Random walk (exercise 1b.2)

Random walk trajectory

The plot of `rw <- cumsum(z_rw)` shows a walk that fluctuates randomly over time. There is no stationary mean, a key property of random walk: variance increases with time.

Different seeds

Different seeds produce different trajectories. Each trajectory is a random walk. Overall, shape looks similar showing that random walks are statistically consistent. With same the seed we obtain the same trajectory, this demonstrates reproducibility.

Larger random walk

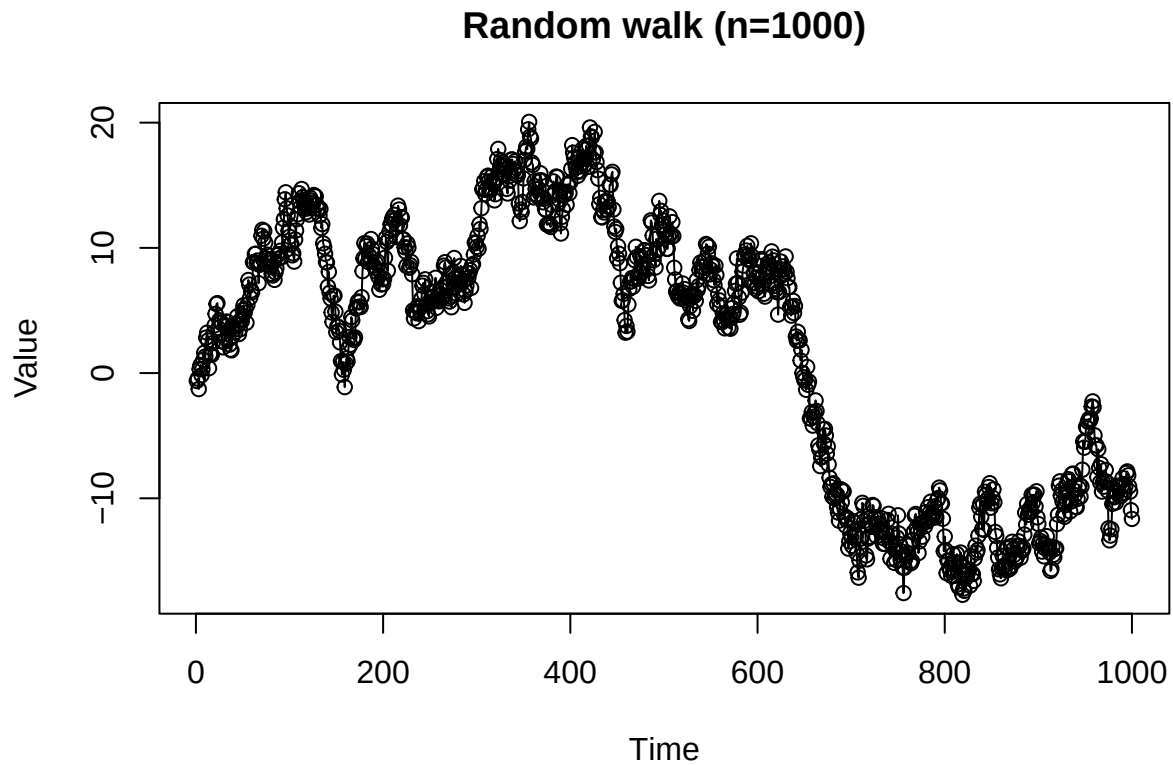
The iterative constructio of `x_alt` with 10000 points produces a longer and smoother path, but still shows random fluctuations. Range of values grows larger with more steps. Variance accumulates over time.

```
# simulate a simple random walk and inspect trajectory and differences
n <- 1000
set.seed(1)
```

```

z_rw <- rnorm(n)
rw <- cumsum(z_rw)
plot(rw, type = "o", main = "Random walk (n=1000)", xlab = "Time", ylab = "Value")

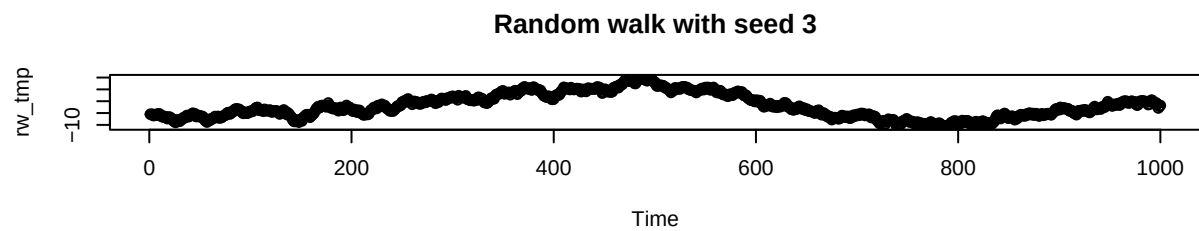
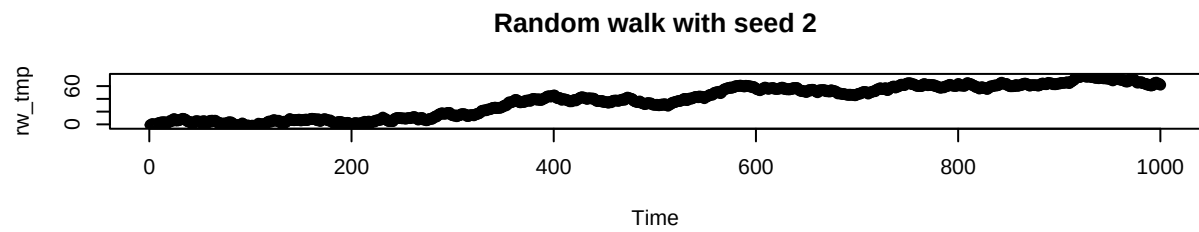
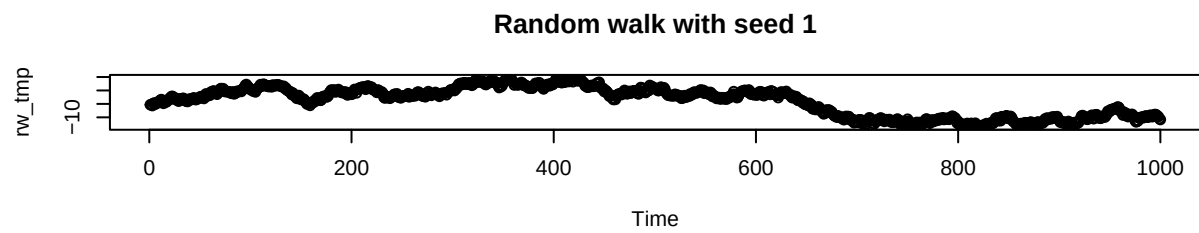
```



```

# replicate with different seeds in a 3-row layout
layout(matrix(1:3, nrow = 3))
for (m in 1:3) {
  set.seed(m)
  z_tmp <- rnorm(n)
  rw_tmp <- cumsum(z_tmp)
  plot(rw_tmp, type = "o", main = paste("Random walk with seed", m), xlab = "Time")
}

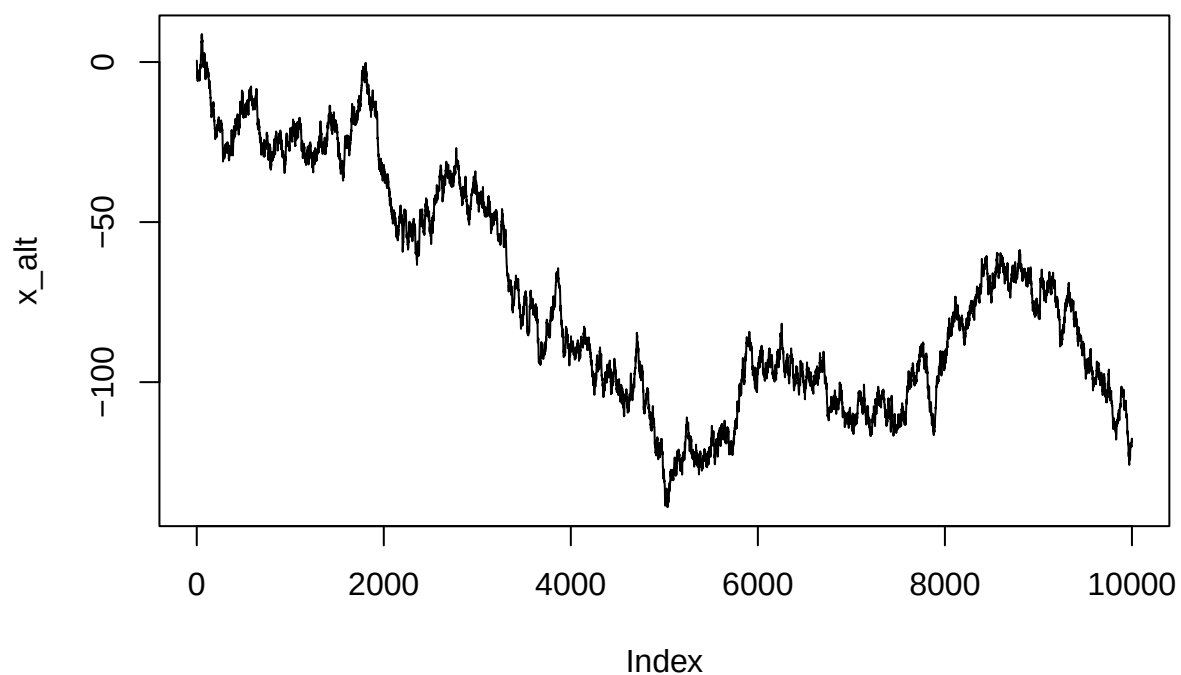
```



```
layout(1)

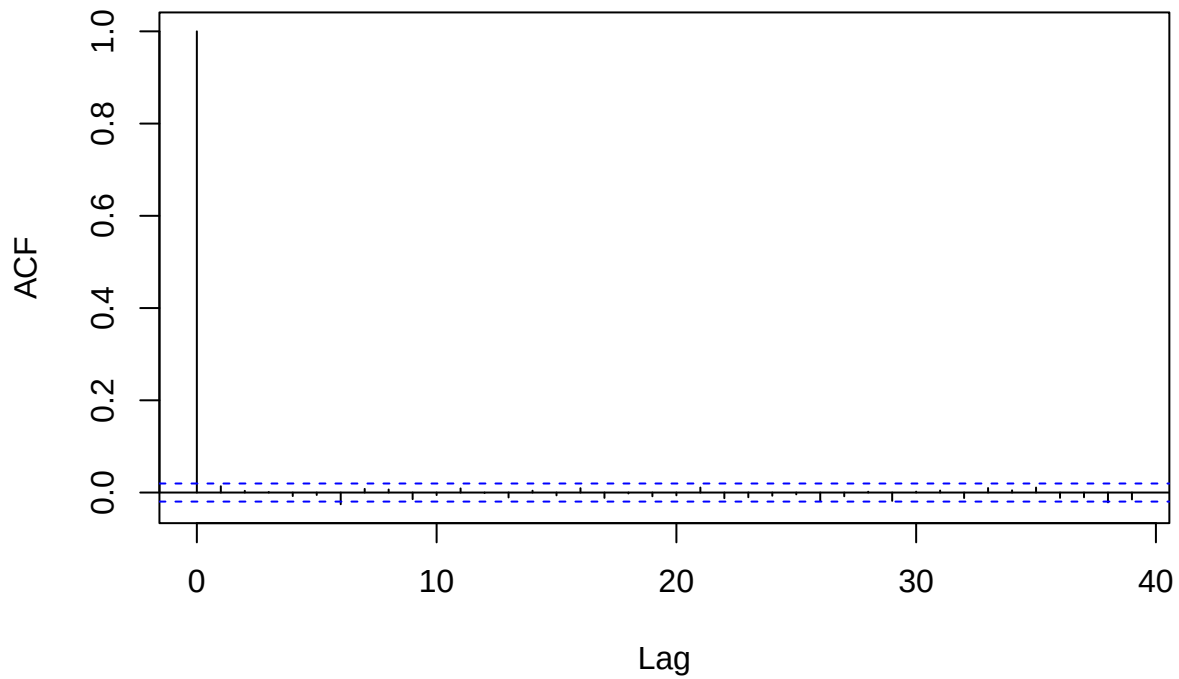
# alternative construction (iterative) for larger n
n2 <- 10000
w <- rnorm(n2)
x_alt <- numeric(n2)
x_alt[1] <- w[1]
for (t in 2:n2) x_alt[t] <- x_alt[t-1] + w[t]
plot(x_alt, type = "l", main = "Random walk (iterative, n=10000)")
```

Random walk (iterative, n=10000)



```
# first difference of a random walk is (approximately) iid noise  
acf(diff(x_alt), main = "Correlogram of first difference of random walk")
```

Correlogram of first difference of random walk



First differences

Taking the first difference of a random walk (`diff(x_alt)`) removes the drift and produces a series that approximates IID Gaussian noise. The correlogram shows no significant autocorrelation.

A key property of random walks is that they are integrated of order 1 ($I(1)$). This means that the first differences gives a stationary series.